Lecture 5: Language Models (II) Seq2Seq & Transformer

Kai-Wei Chang
CS @ UCLA
kw@kwchang.net

Credit: some slides are modified from Vivek Srikumar & Jason Eisner, Chris Manning

Announcement

- Quiz 1 (Grade scope):
  It will be due today
- Finalize and sign up for presentation / final project group
  https://bit.ly/UCLA-CS263-Signup
- Midterm next Monday
- Assignment 1 (BruinLearn)

Decoding algorithms for LMs

Decoding

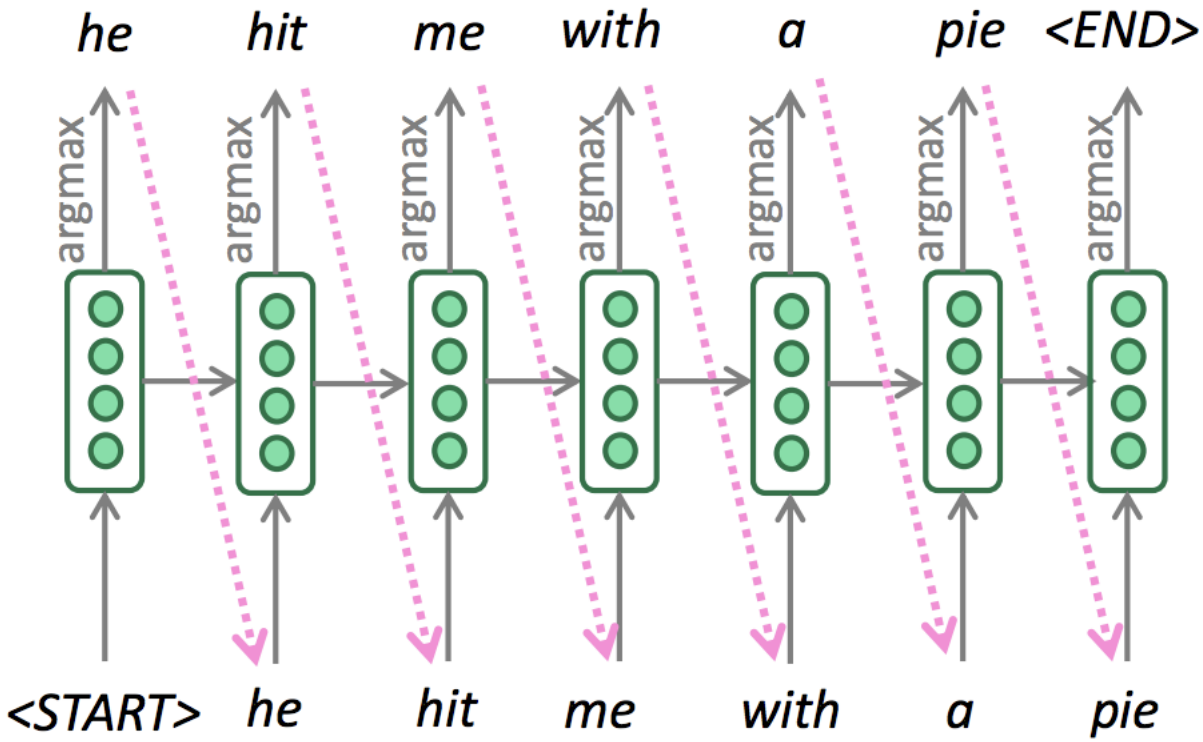
- Ideally, we should enumerate all possible sequences and score them

$$P(y|x) = P(y_1|x)\,P(y_2|y_1,x)\,P(y_3|y_1,y_2,x)\ldots,P(y_T|y_1,\ldots,y_{T-1},x)$$
$$= \prod_{t=1}^{T} P(y_t|y_1,\ldots,y_{t-1},x)$$

- Sequence with length K: $O(|V|^K)$

Decoding

- Greedy decoding



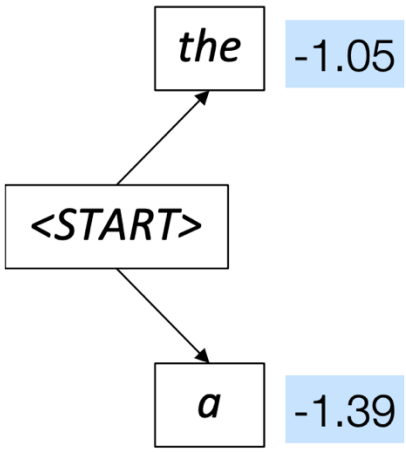
Problem with greedy decoding

Greedy decoding has no way to undo decisions!

- Input: *il a m'entarté* *(he hit me with a pie)*
- → *he* ____
- → *he hit* ____
- → *he hit a* ____ (whoops! no going back now…)

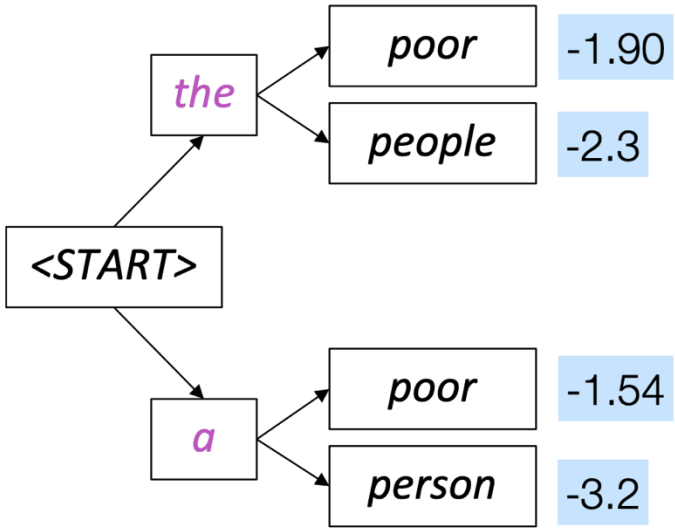
Beam search decoding: example

- Beam size = 2. Showing log-likelihood:



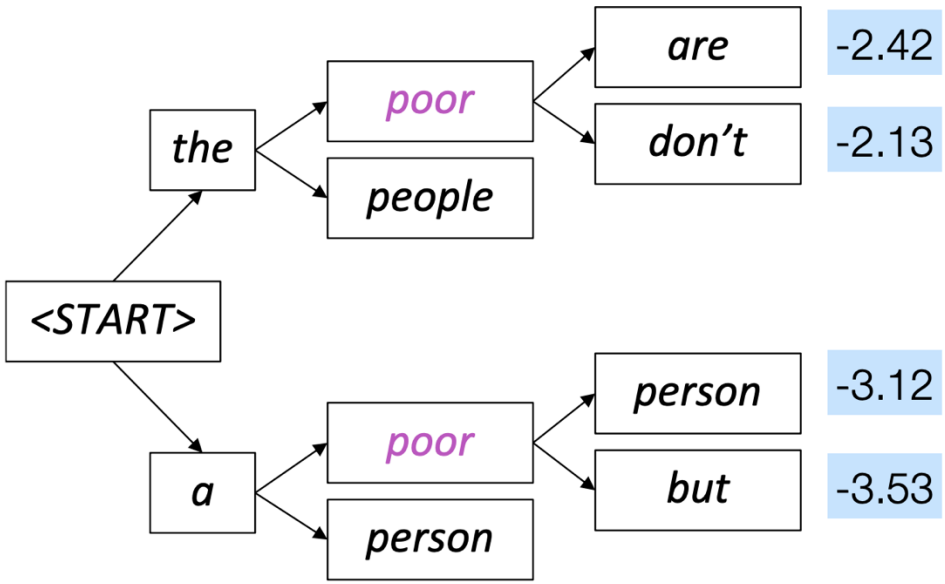
Beam search decoding: example

- Beam size = 2. Showing log-likelihood:



Beam search decoding: example

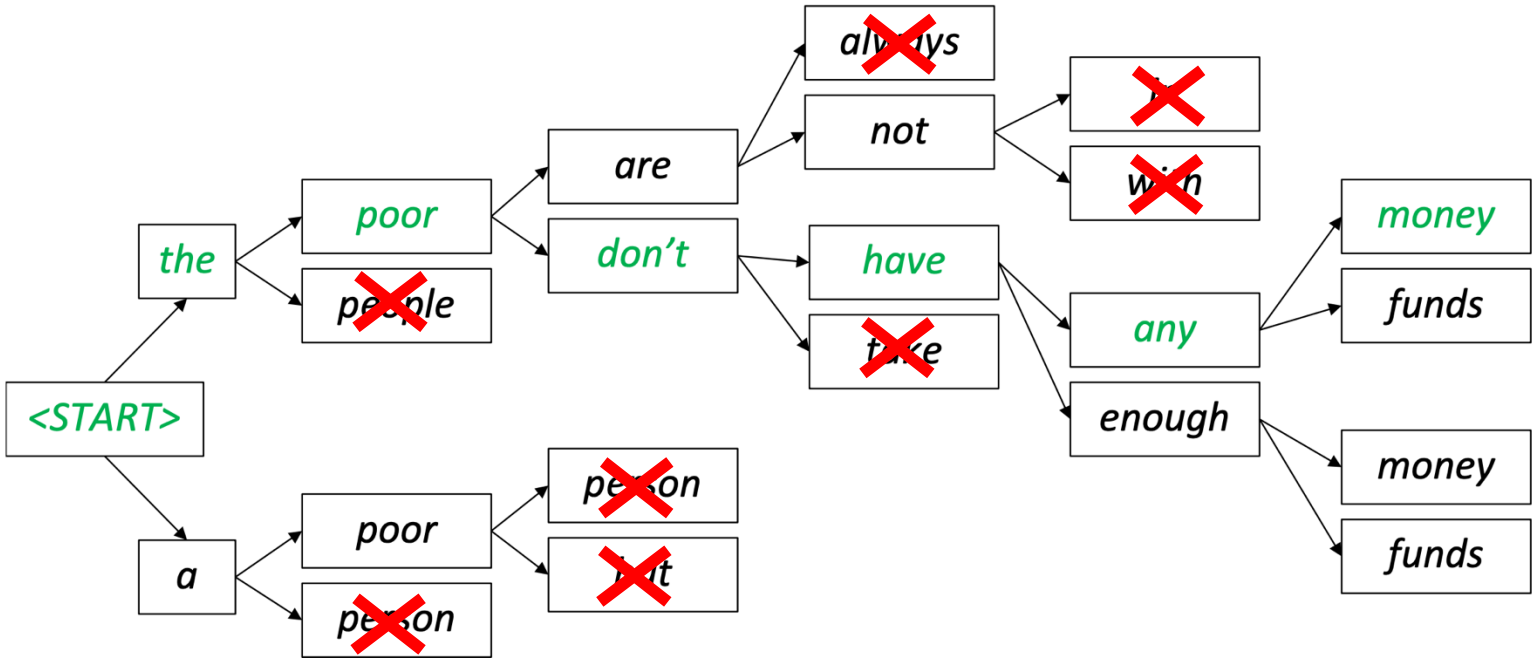
- Beam size = 2. Showing log-likelihood:



Beam search decoding: example

- Beam size = 2. Show log-likelihood:

  A few steps later…..



Exercise

- What beam width makes beam search equivalent to greedy search?

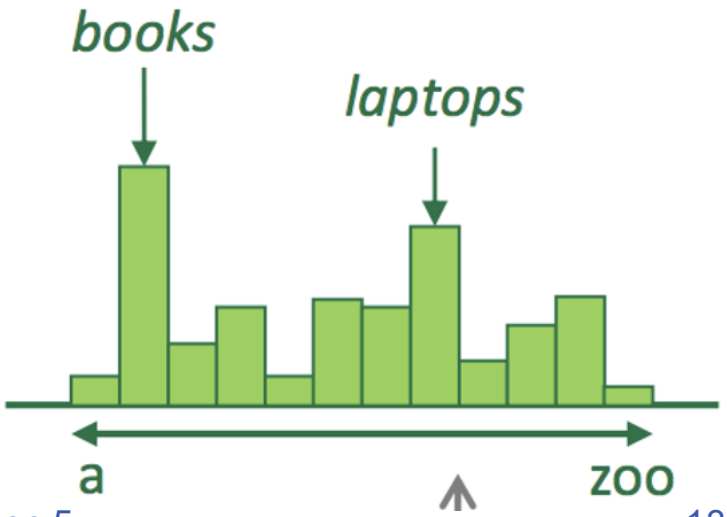
What's the effect of changing beam size k?

- Small k has similar problems to greedy decoding (k=1)
  - Errors in earlier decoding stage leads to wrong answer
- Increasing k:
  - consider more hypotheses
  - more computationally expensive

Sampling-based decoding

- Sampling
  - at each step t, randomly sample from the probability distribution $p(x_t|x_{1:t-1})$ to obtain your next word.
  - Like greedy decoding, but sample instead of argmax.

$$P(x_i|x_{1:i-1}) = \frac{\exp(u_i)}{\sum_j \exp(u_j)}$$



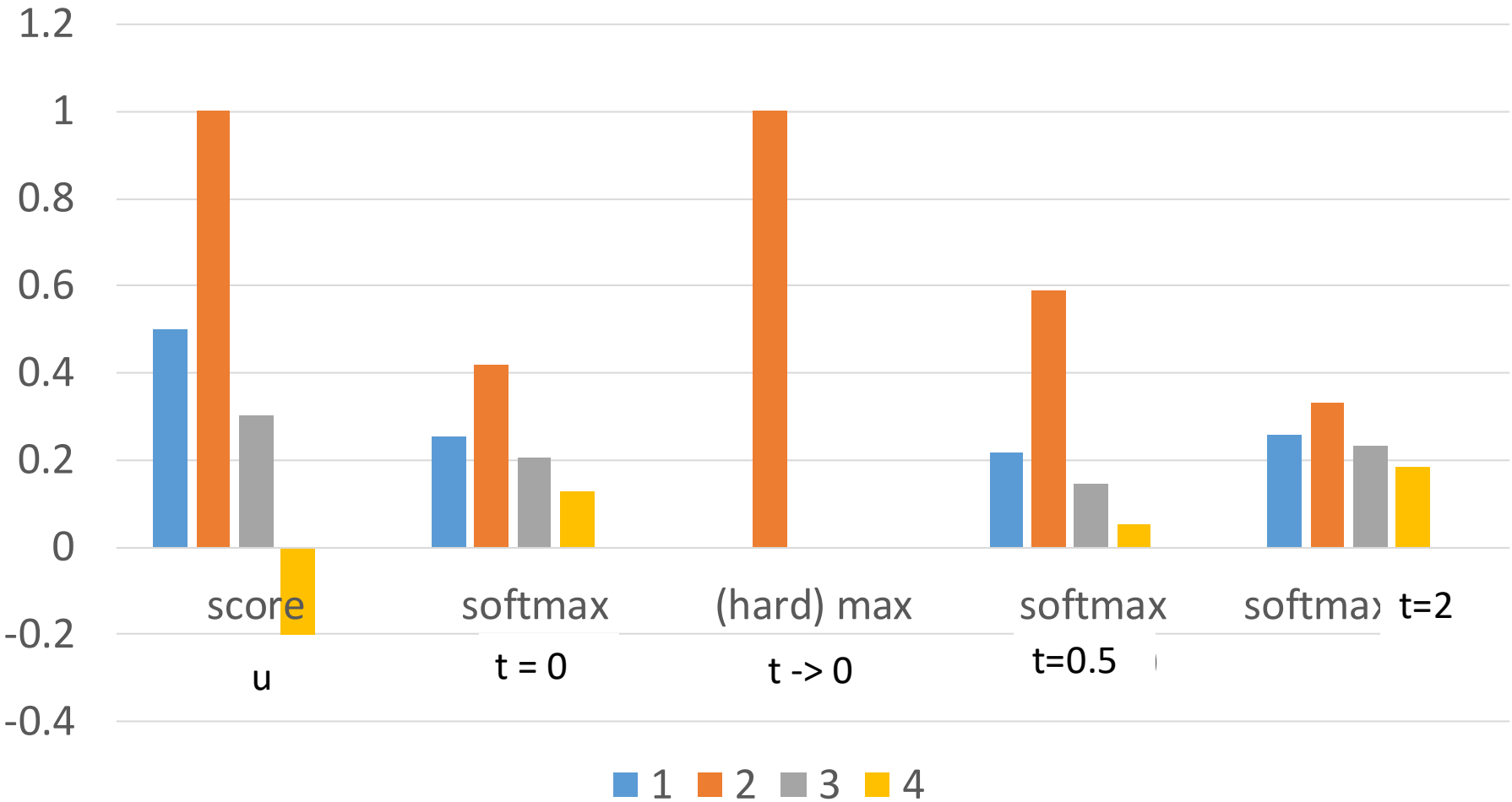
Random Sampling with Temperature

- Temperature T controls the skewness of the distribution

$$P(x_i|x_{1:i-1}) = \frac{\exp(u_i/t)}{\sum_j \exp(u_j/t)}$$

Example

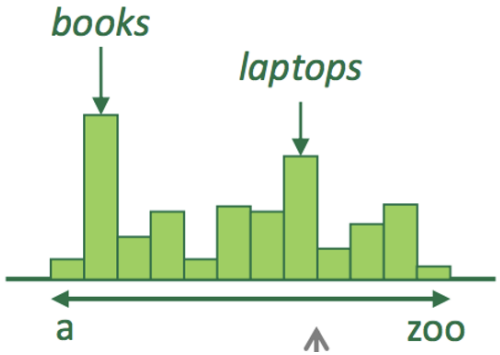
$$P(x_i|x_{1:i-1}) = \frac{\exp(u_i/t)}{\sum_j \exp(u_j/t)}$$



Top-k sampling

- Don't select low-probability words
- Only sample from the top-k words
- K is the hyper-parameter – usually set to 10
- Selecting K may be tricky

- Example K = 2, we sample from {Book, laptops}



Nucleus Sampling

- Select a hyper-parameter p, and find a smallest set of V words such that their sum of their probability is higher than p.

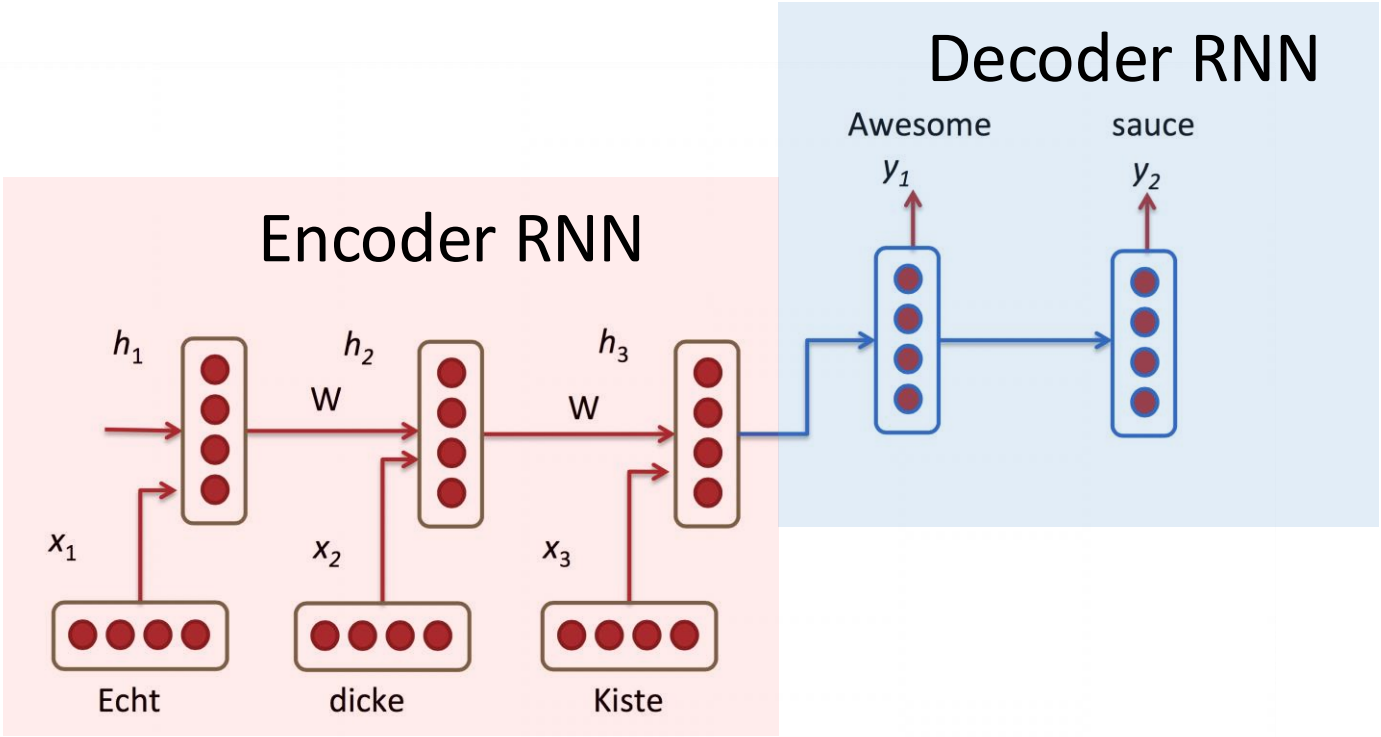
$$\sum_{x \in V^{(p)}} P(x|x_{1:i-1}) \geq p.$$

Decoding algorithms: summary

- Greedy decoding is a simple method; usually gives low quality output
- Beam search (especially with large beam size) searches for high probability output
  - Delivers better quality than greedy, but if beam size is too large, can return high-probability but unsuitable output (e.g. generic, short)
- Sampling methods are a way to get more diversity and randomness
  - Good for open-ended / creative generation (chit-chat, poetry, stories)
  - Top-k and top-p sampling allows you to control diversity

Seq2Seq Models

Seq2seq (Sutskever et al., 2014)



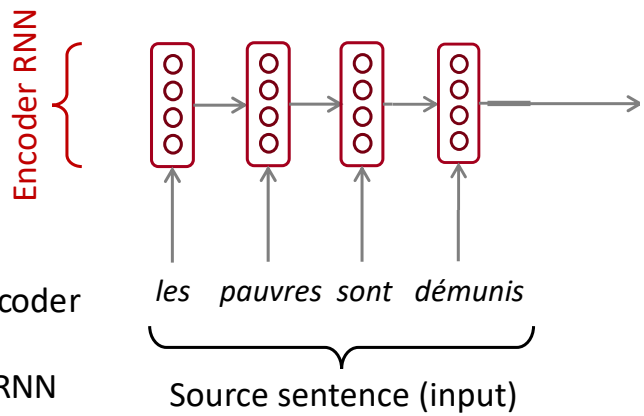
Source: http://www.wildml.com/2015/09/recurrent-neural-networks-tutorial-part-1-introduction-to-rnns

From RNN-LM to MT

- Encode the source sentence into a vector
  - This vector is the product of an RNN (encoder RNN)
  - What do we take from the encoder RNN to be the sentence vector?
- Feed the sentence vector to <u>another RNN</u> (the decoder)
  - Where should this vector go?
- We'll train them together in what's called the sequence-to-sequence (seq2seq) or encoder-decoder model more generally.

Neural Machine Translation (NMT)

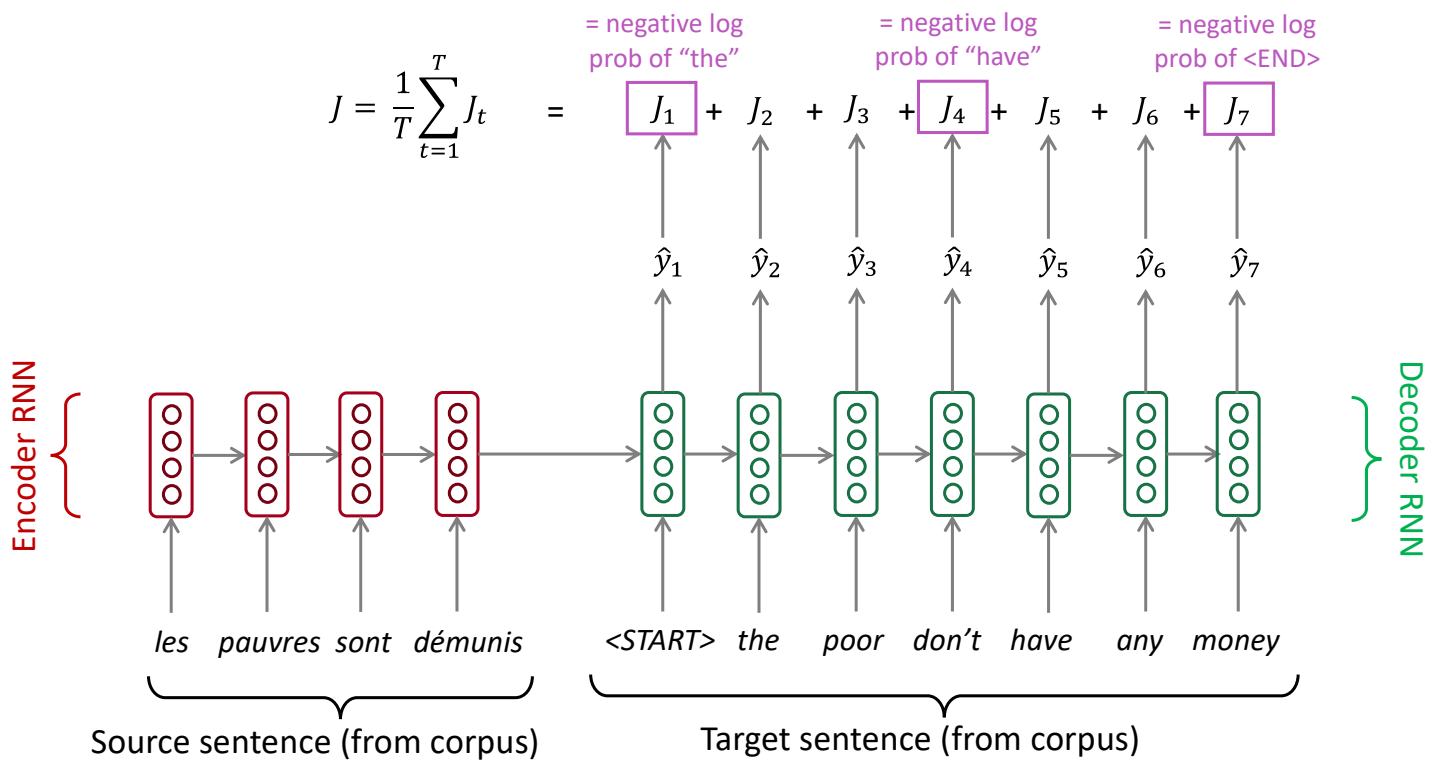
The sequence-to-sequence model



Extension: encoder often a <u>bidirectional</u> RNN (two RNNs working in opposite directions)

What are the benefits?

Training a Neural Machine Translation system



Seq2seq is optimized as a **single system.**
Backpropagation operates *"end to end"*.

From Language Model to Translation Model

- An RNN LM estimates
  $$p(\boldsymbol{y} = y_1, y_2, \dots, y_n)$$
  $$= p(y_1)p(y_2|y_1) \dots p(y_n|y_1, y_2, \dots, y_{n-1})$$
- A seq2seq TM also estimates probability of **y** but conditions it on an input sequence **x**. It's a <u>conditional language model</u>
  $$p(\boldsymbol{y}|\boldsymbol{x})$$
  $$= p(y_1|\boldsymbol{x})p(y_2|y_1, \boldsymbol{x}) \dots p(y_n|y_1, y_2, \dots, y_{n-1}, \boldsymbol{x})$$
- Trained as one big RNN (with two sets of parameters switch in the middle), basically in the same way as RNN-LM

Exercise

What is the role of the encoder in a seq2seq model?

A. Generate output tokens
B. Convert input sequence into a fixed representation
C. Calculate loss
D. Apply softmax

Seq2Seq with Attention

Motivated by the MT task

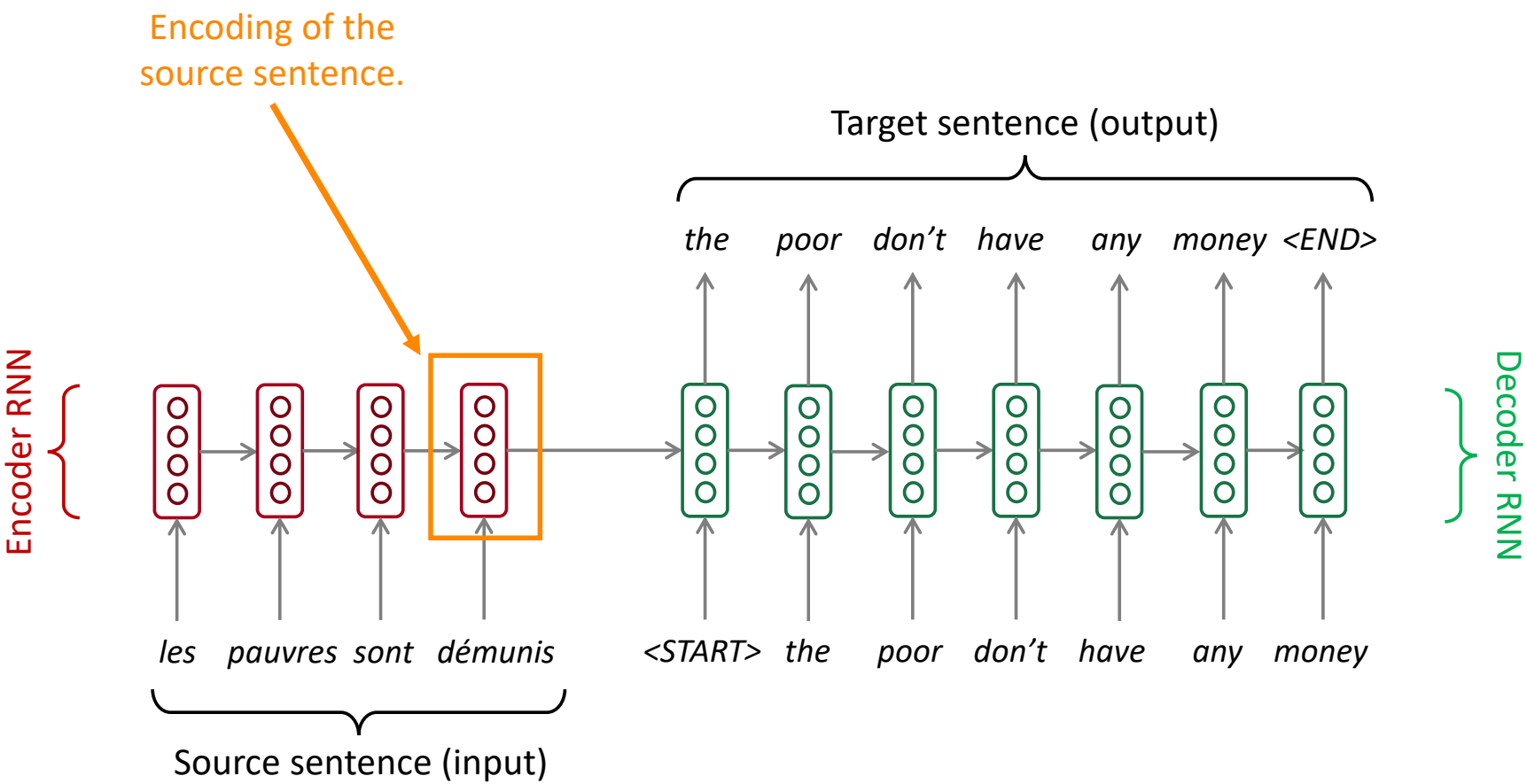
NMT is the **flagship task** for NLP Deep Learning

- NMT research has pioneered many of the recent innovations of NLP Deep Learning

- In **2018**: NMT research continues to thrive
  - Researchers have found *many, many* improvements to the "vanilla" seq2seq NMT system we've presented today
  - But one improvement is so integral that it is the new vanilla…

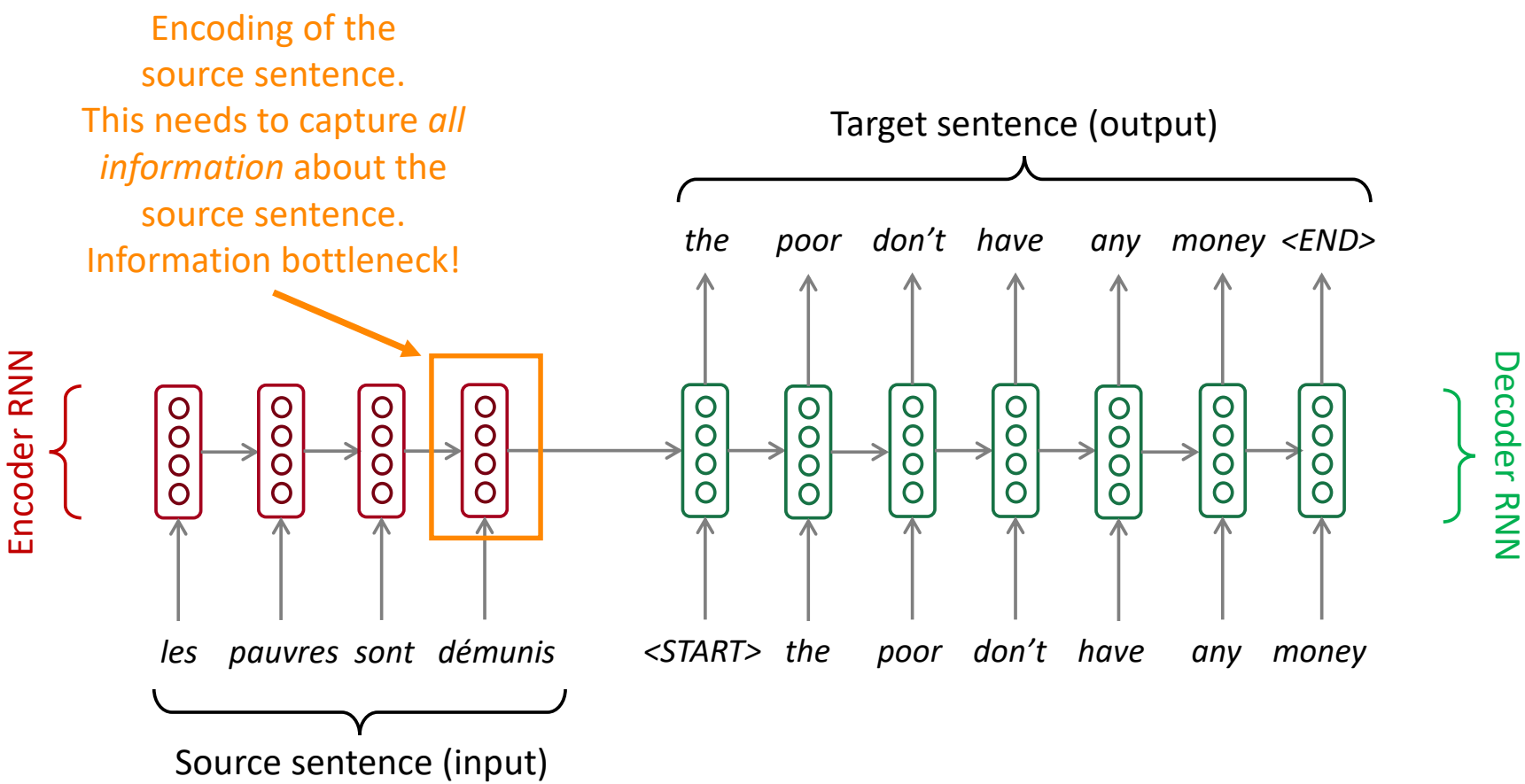
ATTENTION

Sequence-to-sequence: the bottleneck problem

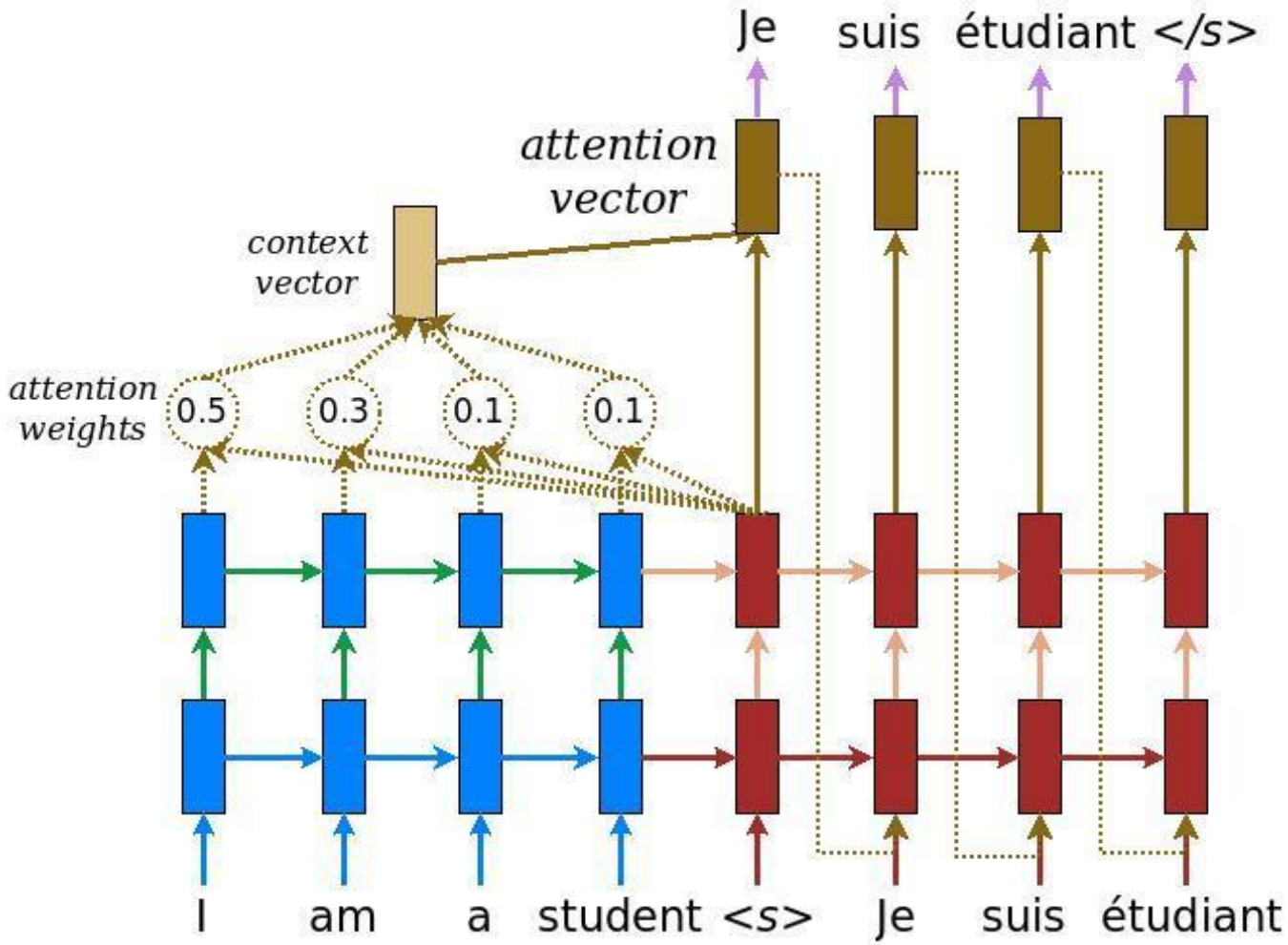


Problems with this architecture?

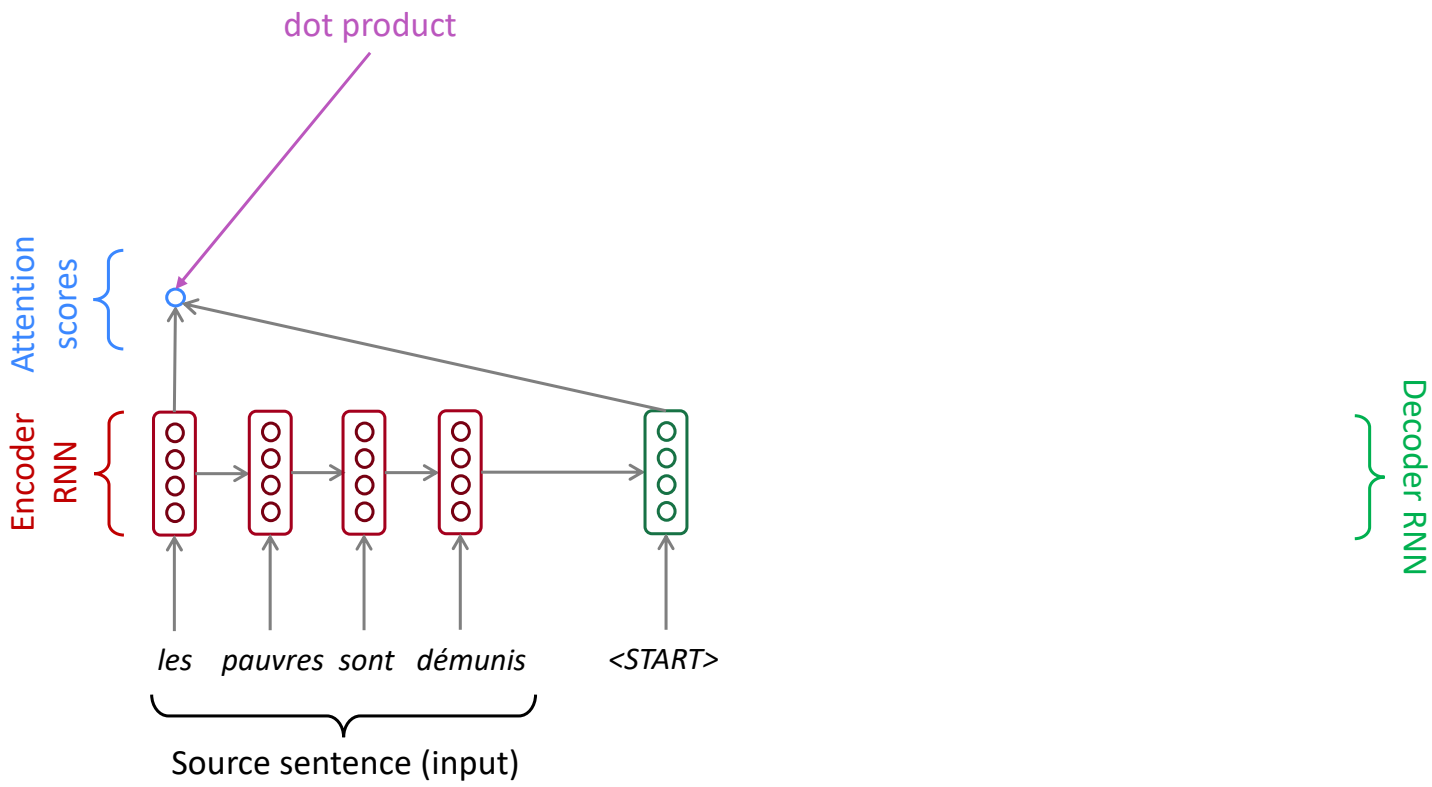
Sequence-to-sequence: the bottleneck problem



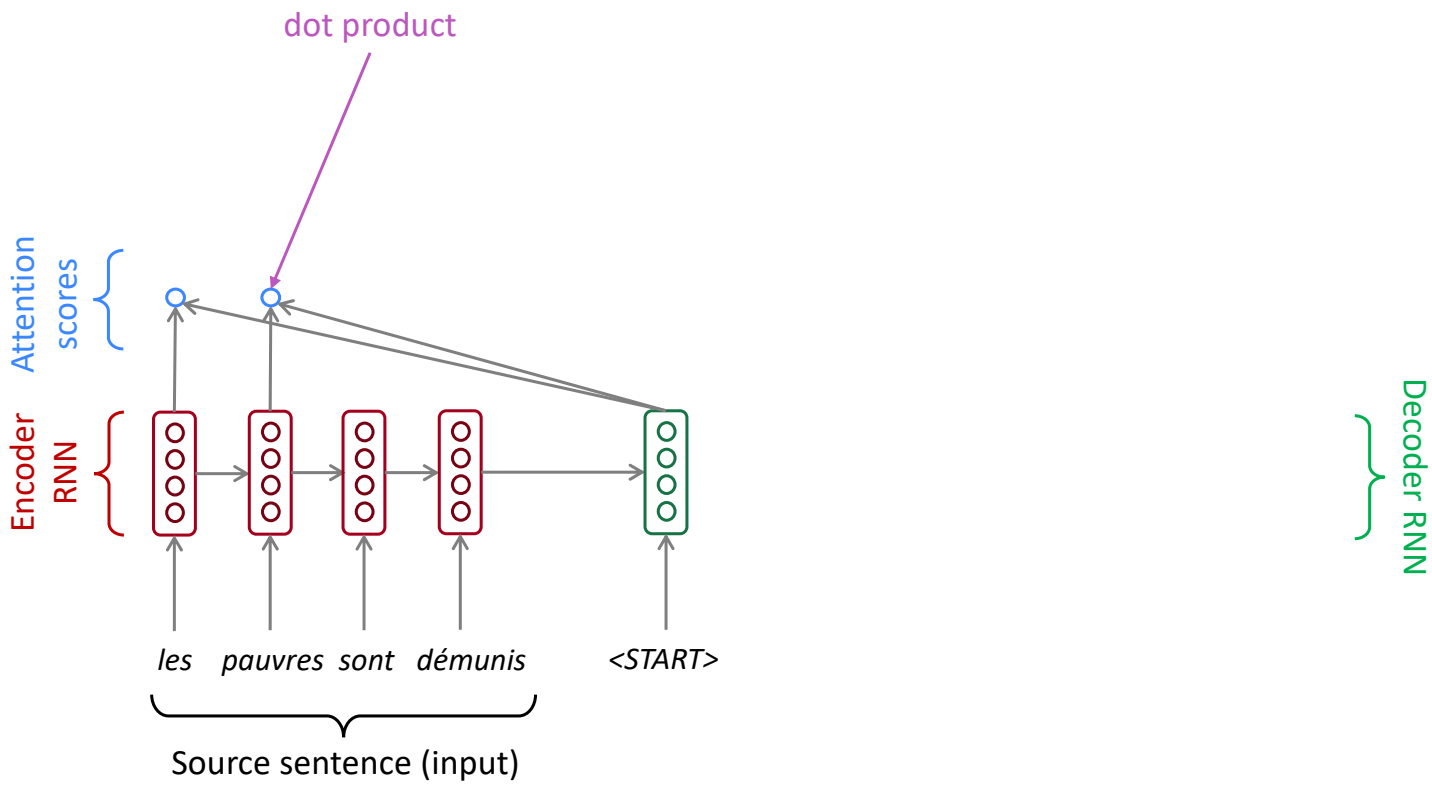
Seq2Seq with Attention



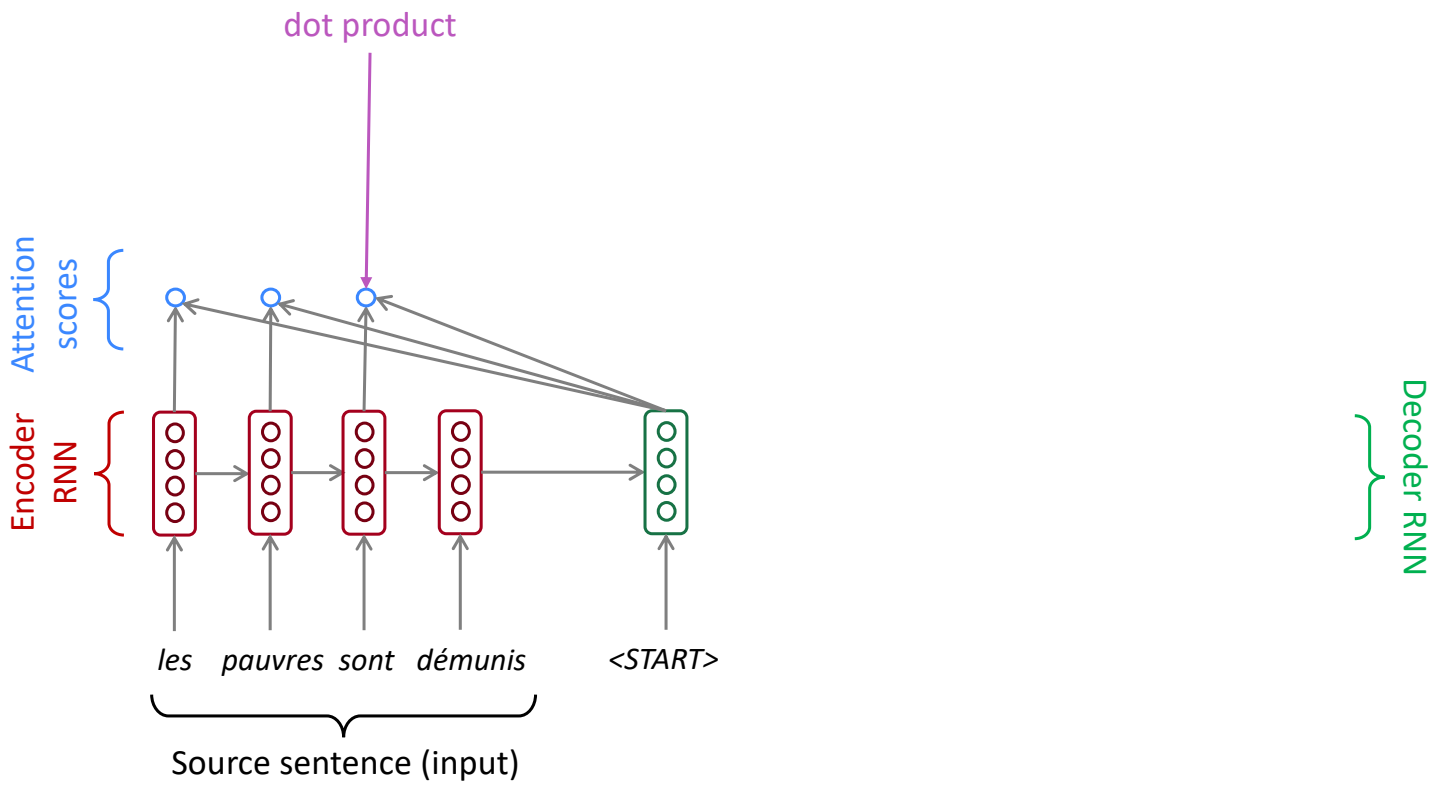
Sequence-to-sequence with attention



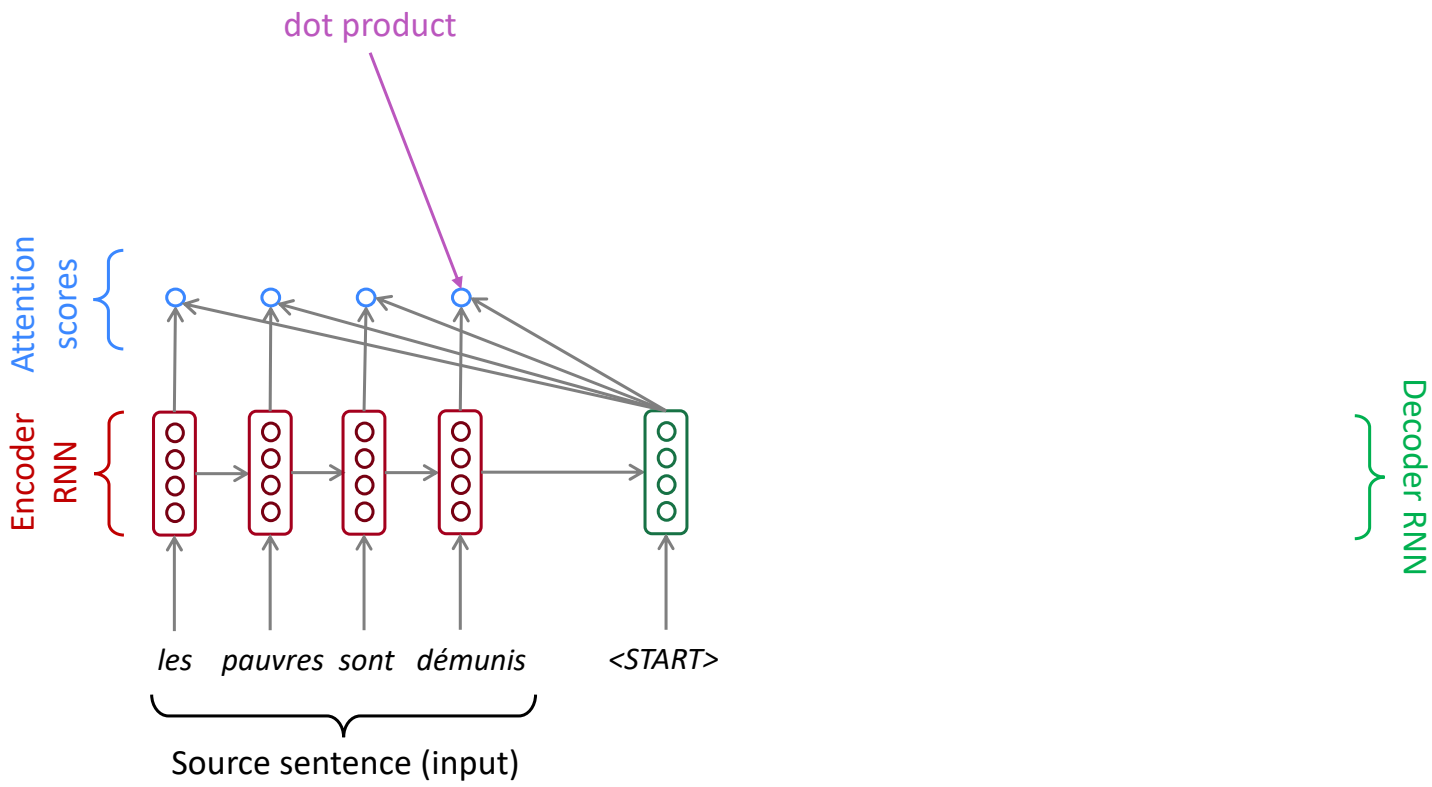
Sequence-to-sequence with attention



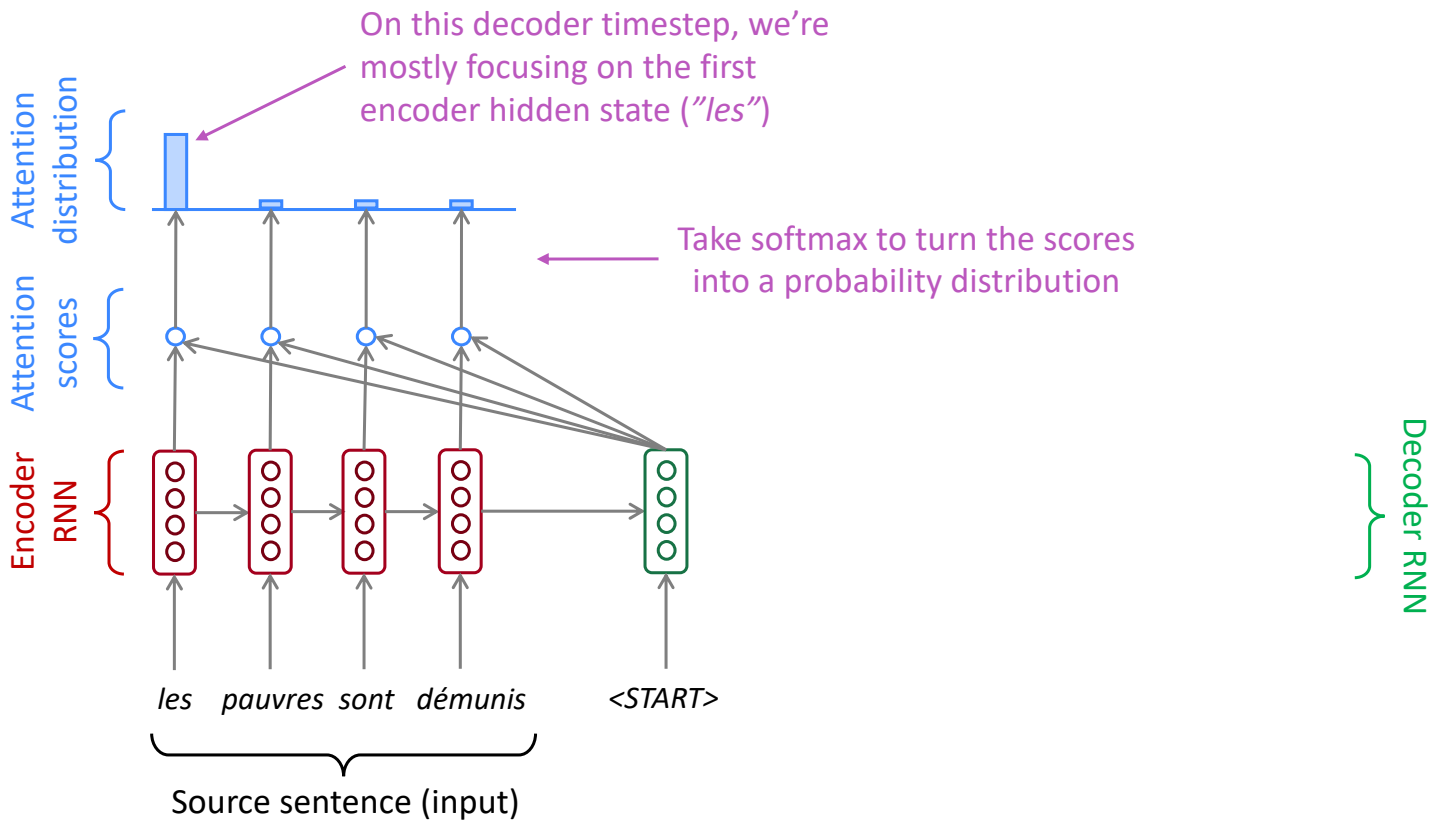
Sequence-to-sequence with attention



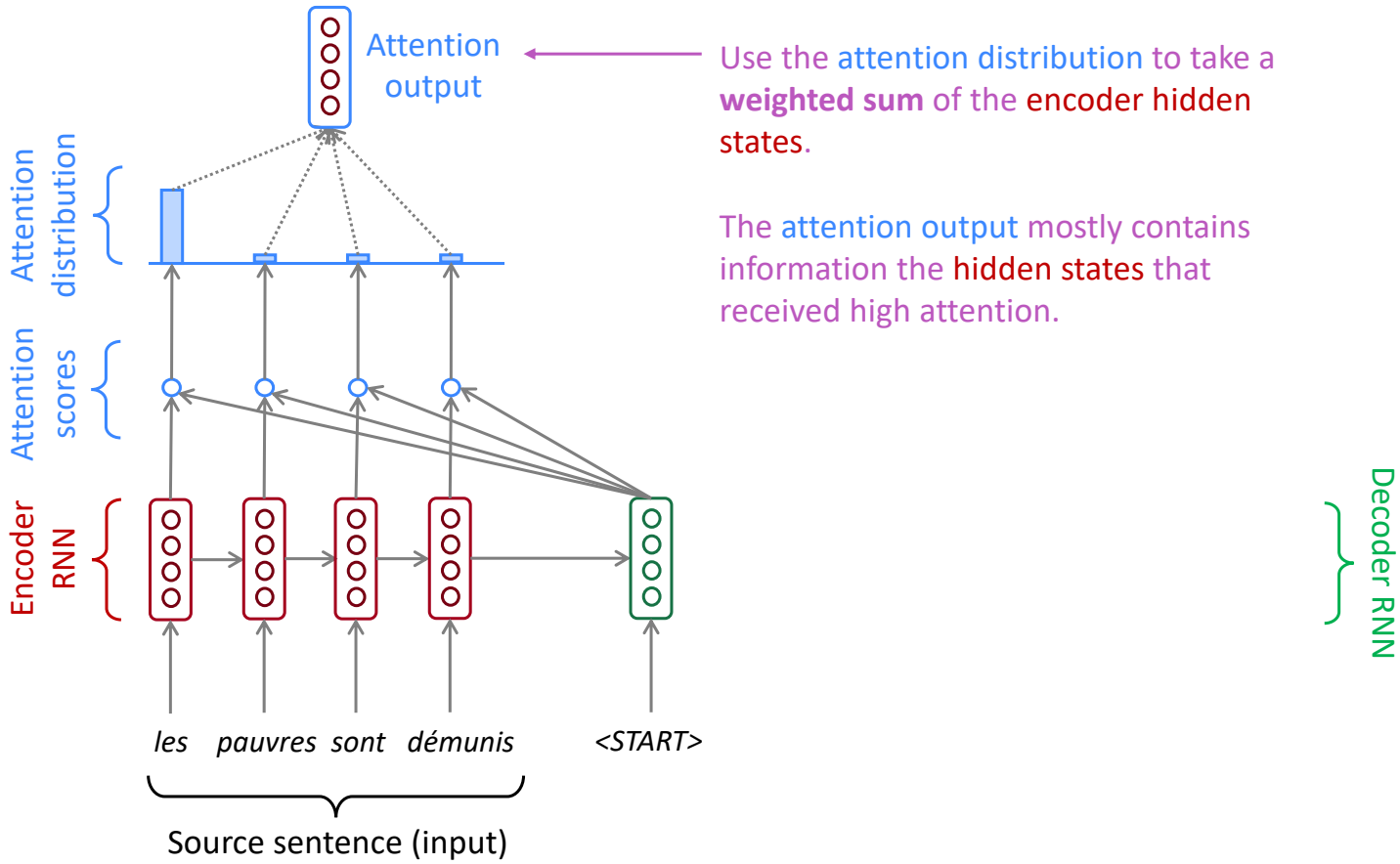
Sequence-to-sequence with attention



Sequence-to-sequence with attention



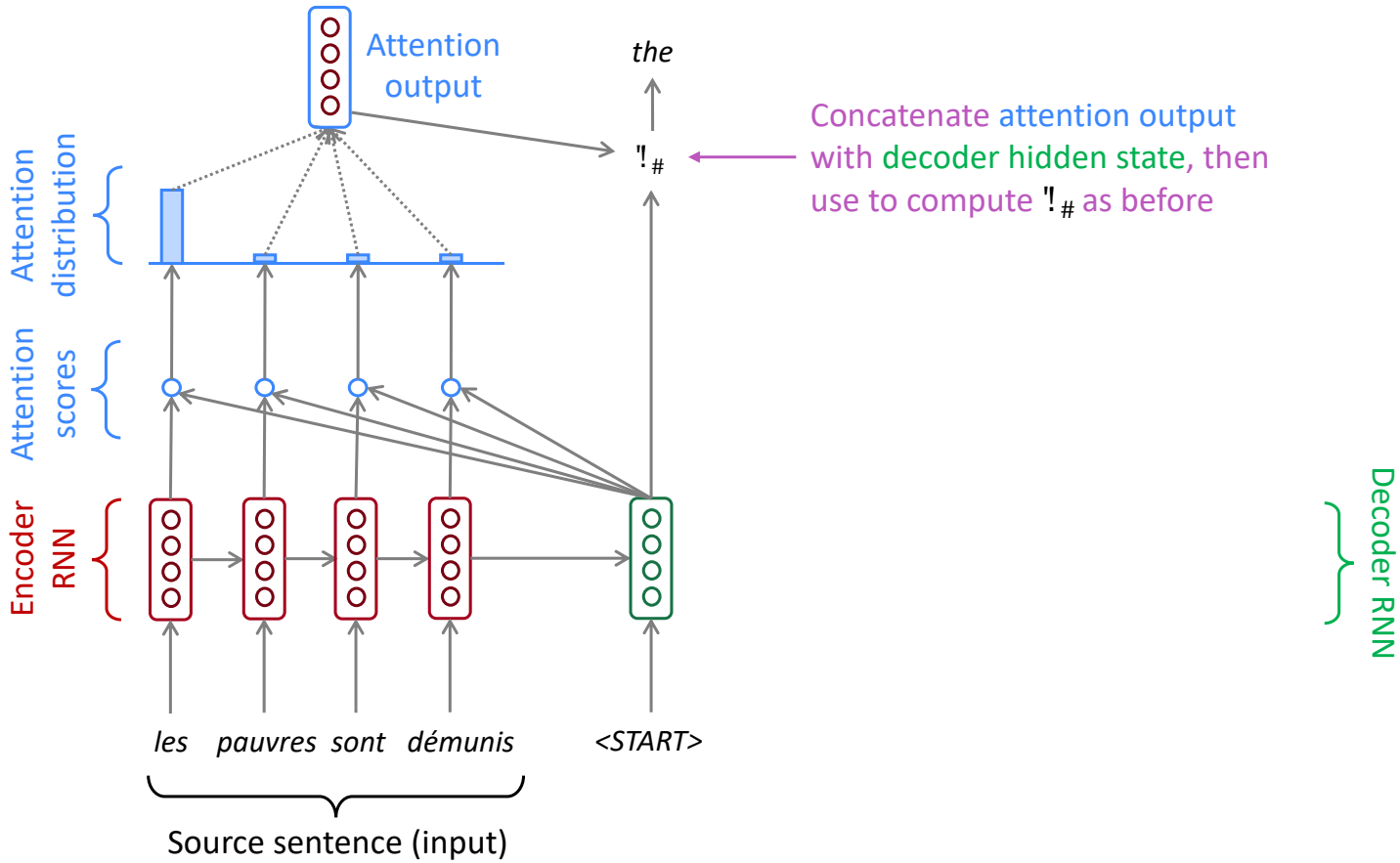
Sequence-to-sequence with attention



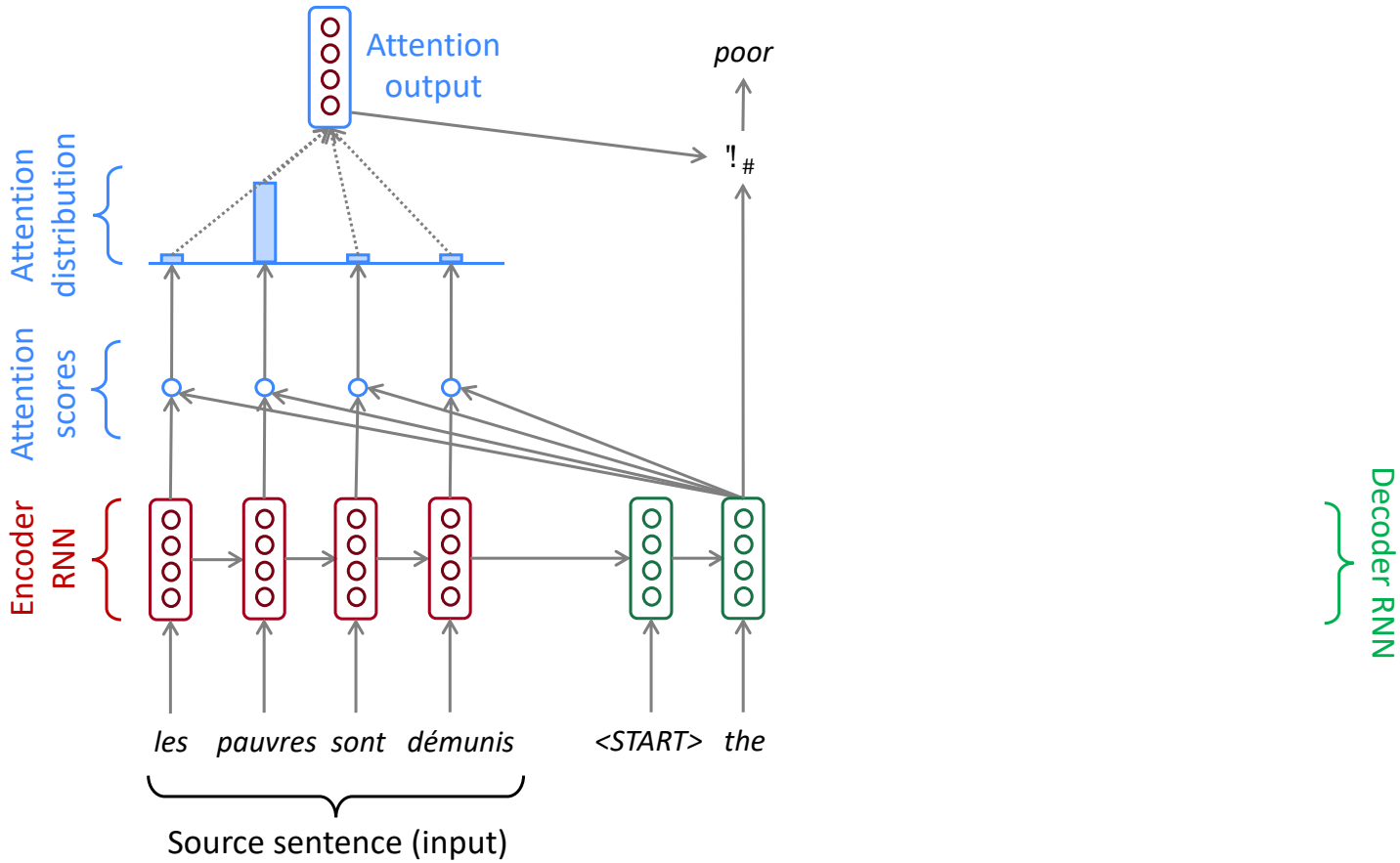
Use the attention distribution to take a **weighted sum** of the encoder hidden states.

The attention output mostly contains information the hidden states that received high attention.

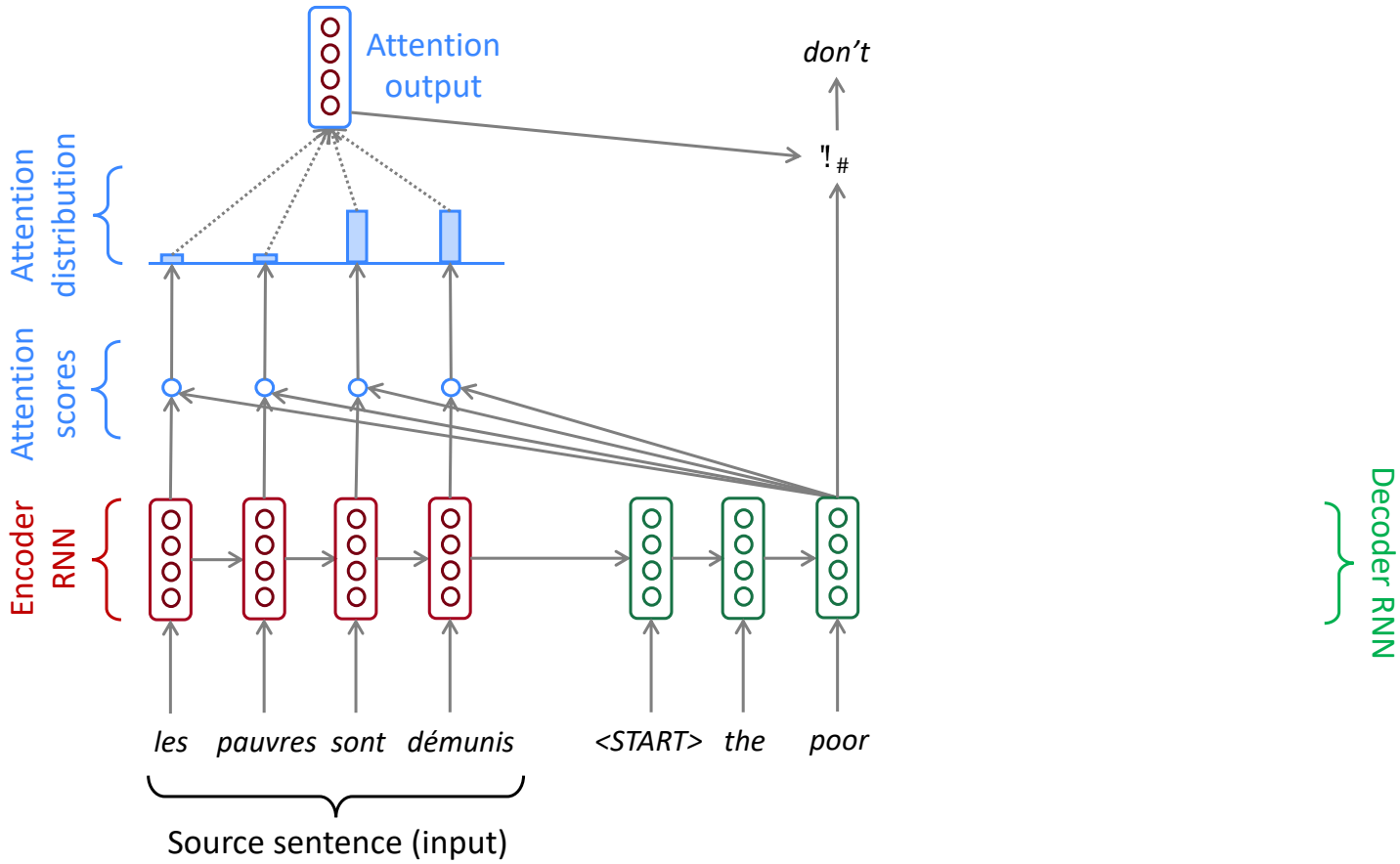
Sequence-to-sequence with attention



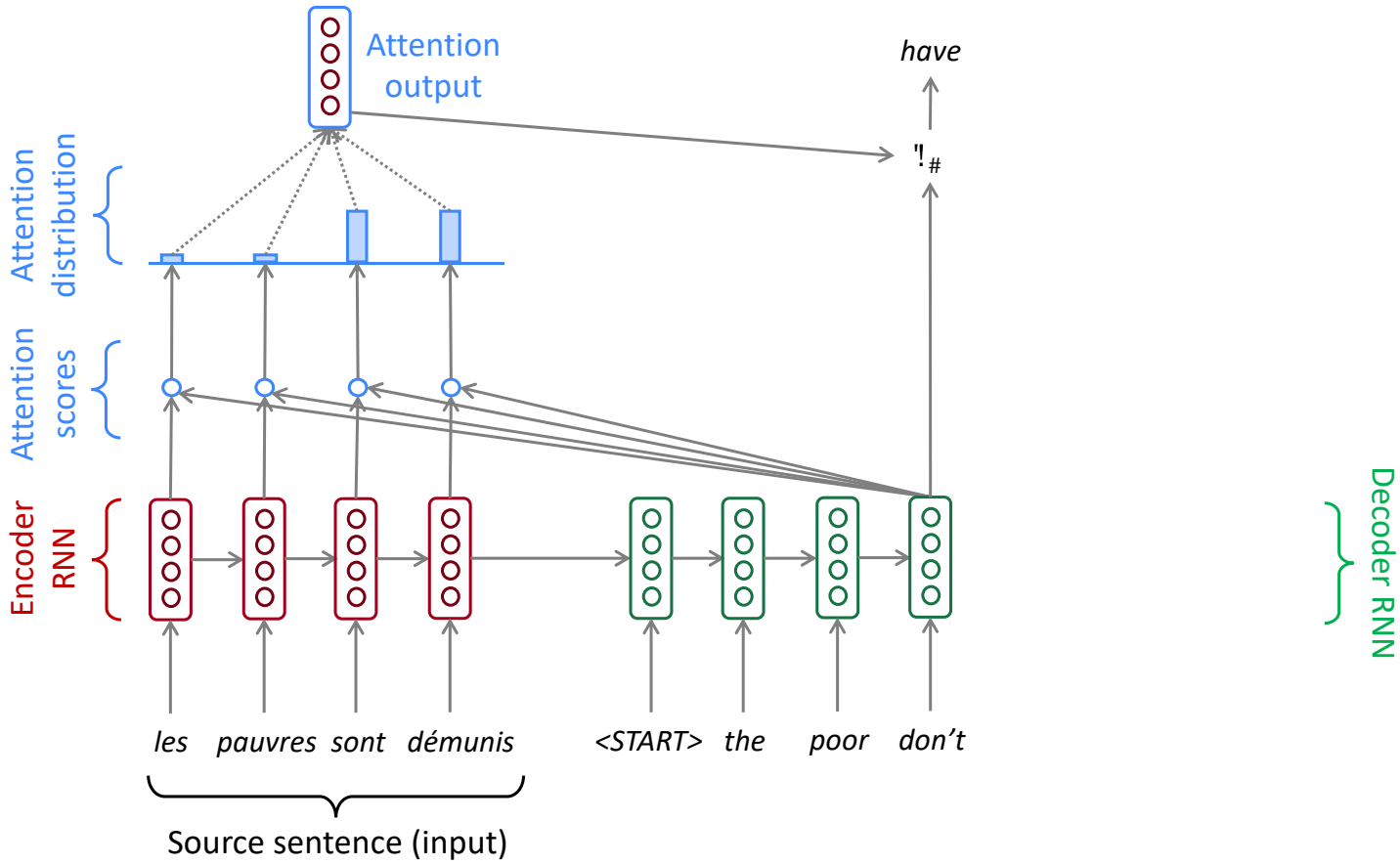
Sequence-to-sequence with attention



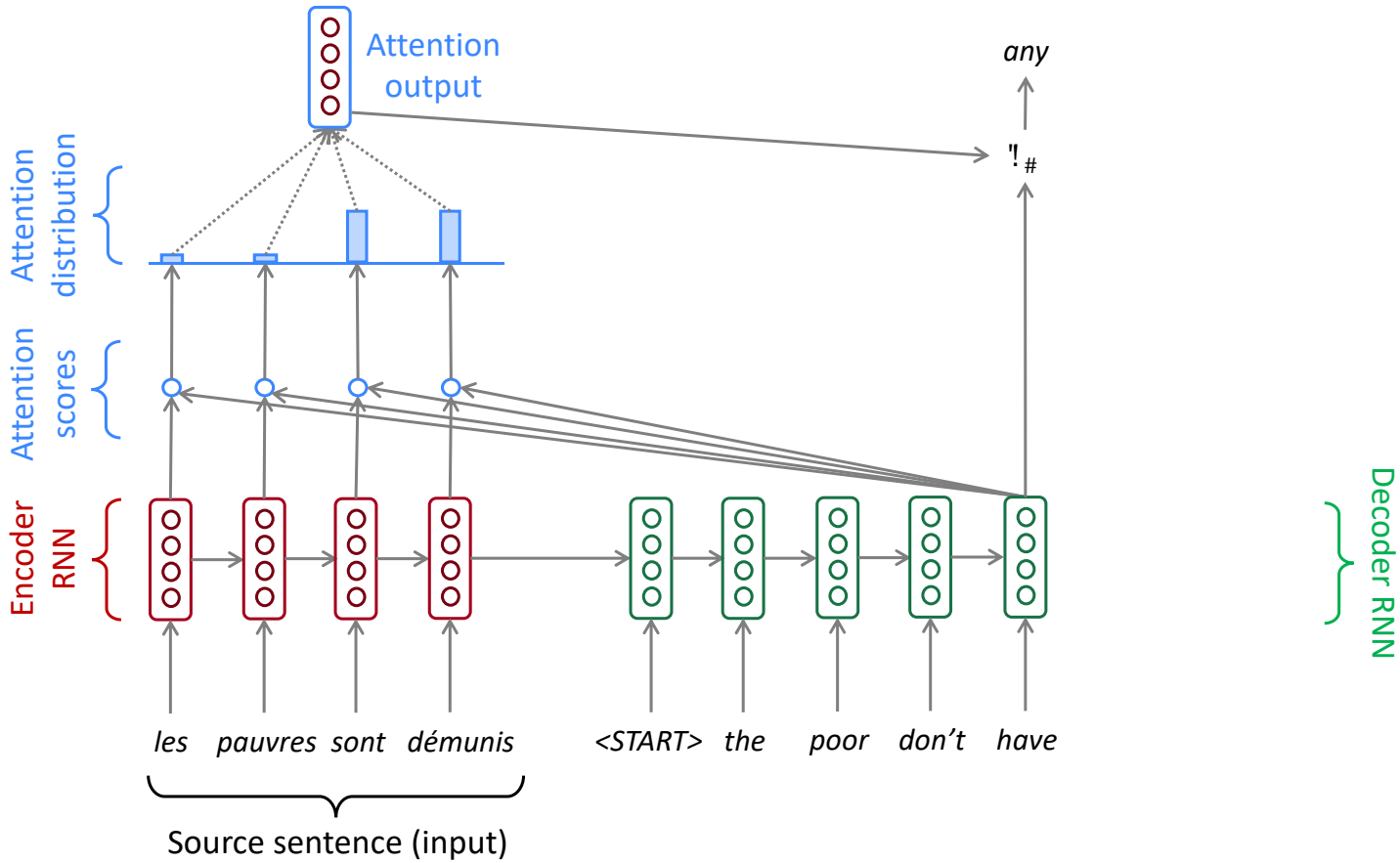
Sequence-to-sequence with attention



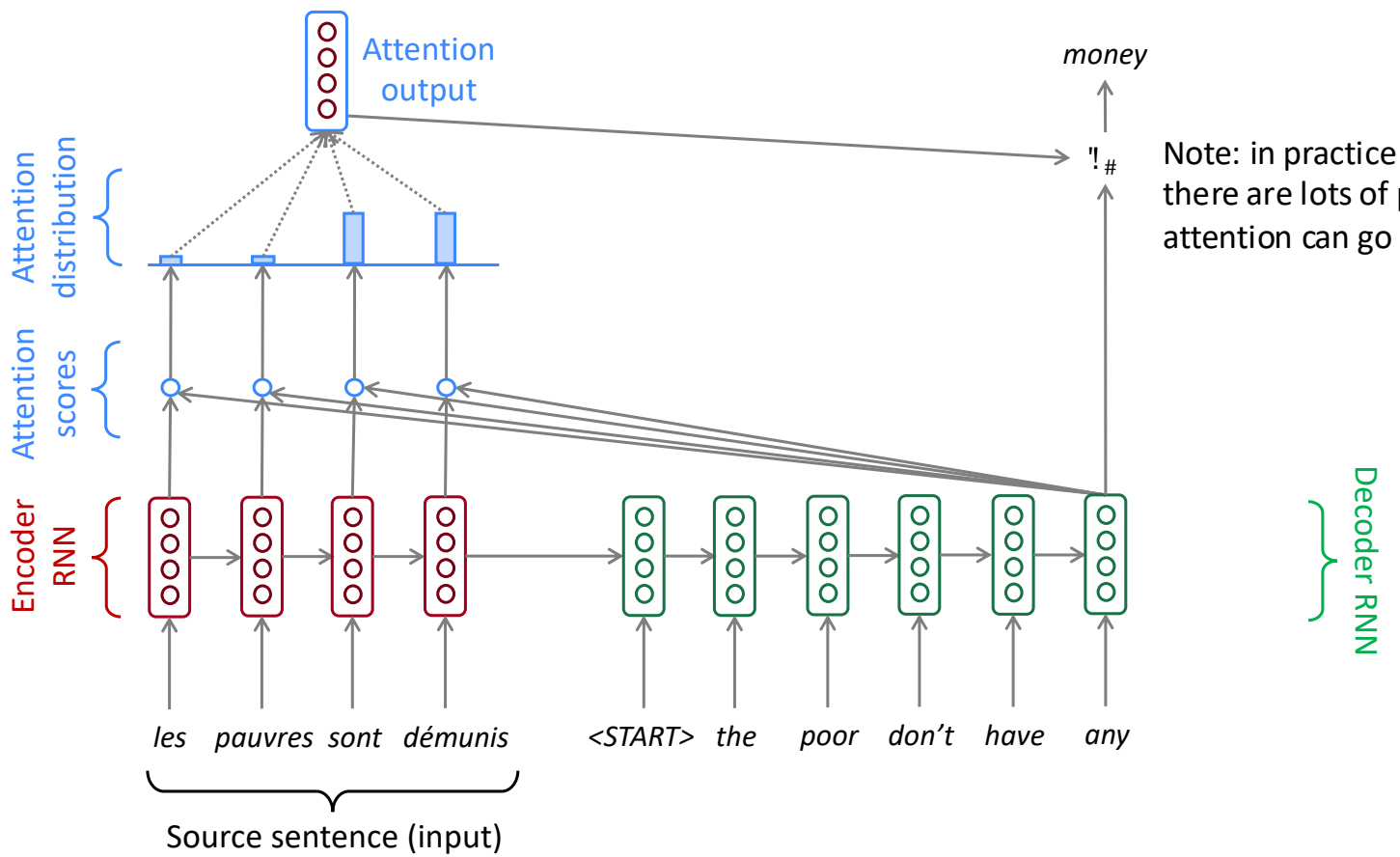
Sequence-to-sequence with attention



Sequence-to-sequence with attention



Sequence-to-sequence with attention



Note: in practice there are lots of places attention can go in the networks

Attention in equations

- We have encoder hidden states $[\boldsymbol{h}_s^1, \boldsymbol{h}_s^2, \boldsymbol{h}_s^3, \ldots \boldsymbol{h}_s^N]$, $\boldsymbol{h}_s^t \in R^h$
- At decoding timestep t, we have decoder hidden state $\boldsymbol{h}_d^t$
- We get the attention scores *for this step* as:
  - $\boldsymbol{a}^t = [sim(\boldsymbol{h}_d^t, \boldsymbol{h}_s^1), sim(\boldsymbol{h}_d^t, \boldsymbol{h}_s^2), sim(\boldsymbol{h}_d^t, \boldsymbol{h}_s^3) \ldots sim(\boldsymbol{h}_d^t, \boldsymbol{h}_s^N)] \in R^N$
- Take softmax to get the final attention distribution
  - $\boldsymbol{\alpha}^t = \text{softmax}(\boldsymbol{a}^t) \in R^N$
- We use $\boldsymbol{\alpha}^t$ to get the weighted sum of the encoder hidden states to get attention outputs
  - $\boldsymbol{c}^t = \sum_{i=1}^{N} \alpha_i^t \, \boldsymbol{h}_s^i \in R^h$
- We use $[\boldsymbol{c}^t; \boldsymbol{h}_d^t]$ instead of $\boldsymbol{h}_d^t$ to predict the output.

Attention in equations (cont.)

- Two popular methods to compute the attention score:
  - **Multiplicative** (Luong et al., 2015)

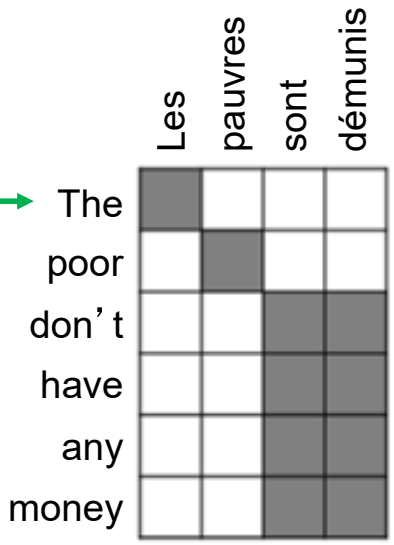
$$\widetilde{\alpha}_{ji} = \boldsymbol{h}_j^{(\mathrm{tar})} W^T \boldsymbol{h}_i^{(\mathrm{src})}$$

  - **Additive** (Bahdanau et al., 2014)

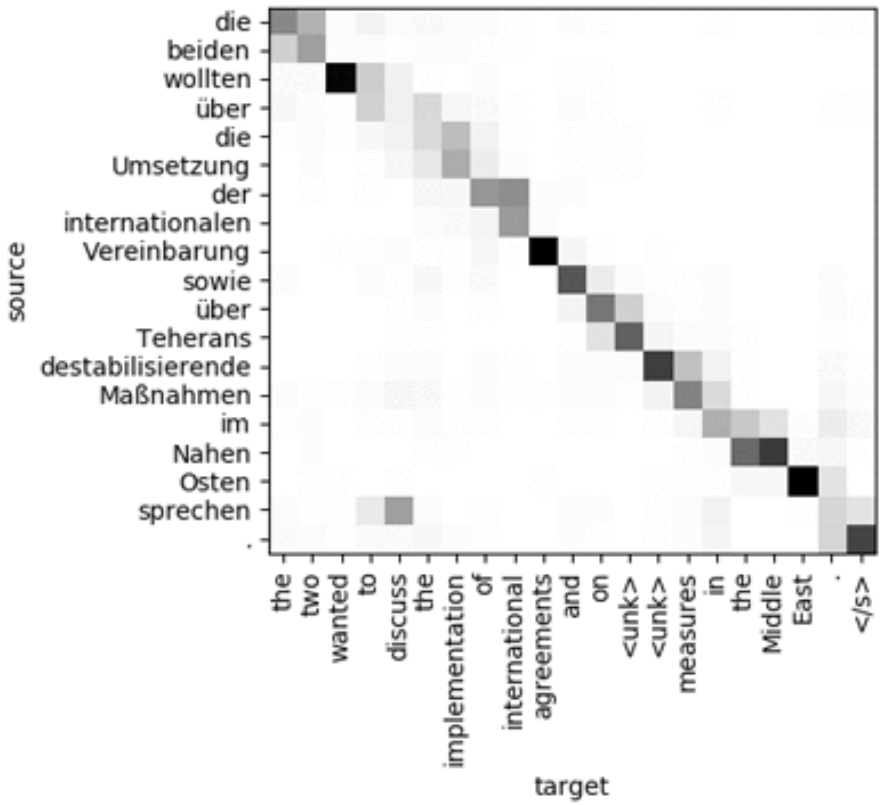
$$\widetilde{\alpha}_{ji} = v_a^T \tanh\left(W_1 \boldsymbol{h}_j^{(\mathrm{tar})} + W_2 \boldsymbol{h}_i^{(\mathrm{src})}\right)$$

Attention is great!

- Attention significantly improves NMT performance
  - It's very useful to allow decoder to focus on certain parts of the source
- Attention solves the bottleneck problem
  - Attention allows decoder to look directly at source; bypass bottleneck
- Attention helps with vanishing gradient problem
  - Provides shortcut to faraway states
- Attention provides some interpretability
  - By inspecting attention distribution, we can see what the decoder was focusing on
  - We get alignment for free!
  - This is cool because we never explicitly trained an alignment system
  - The network just learned alignment by itself



Visualizing Attention in Machine Translation



Source: https://aws.amazon.com/blogs/machine-learning/train-neural-machine-translation-models-with-sockeye/

Exercise

- **How does attention improve seq2seq models?**

  A. Removes the encoder

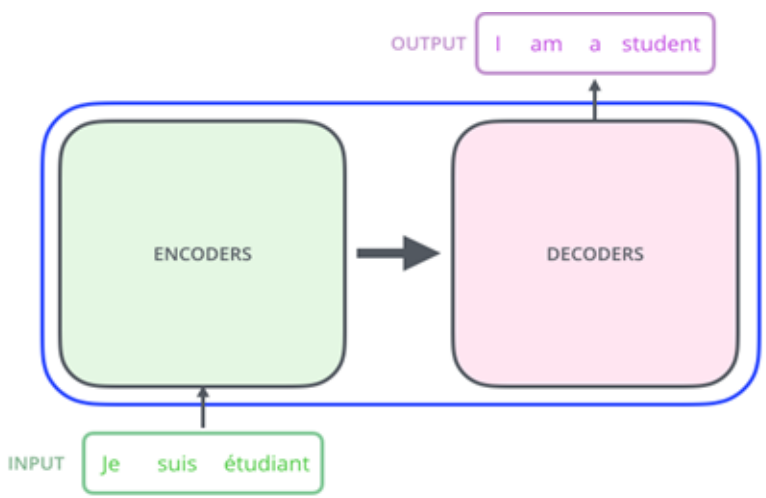
  B. Allows decoder to focus on relevant parts of input

  C. Reduces model size

  D. Eliminates need for training

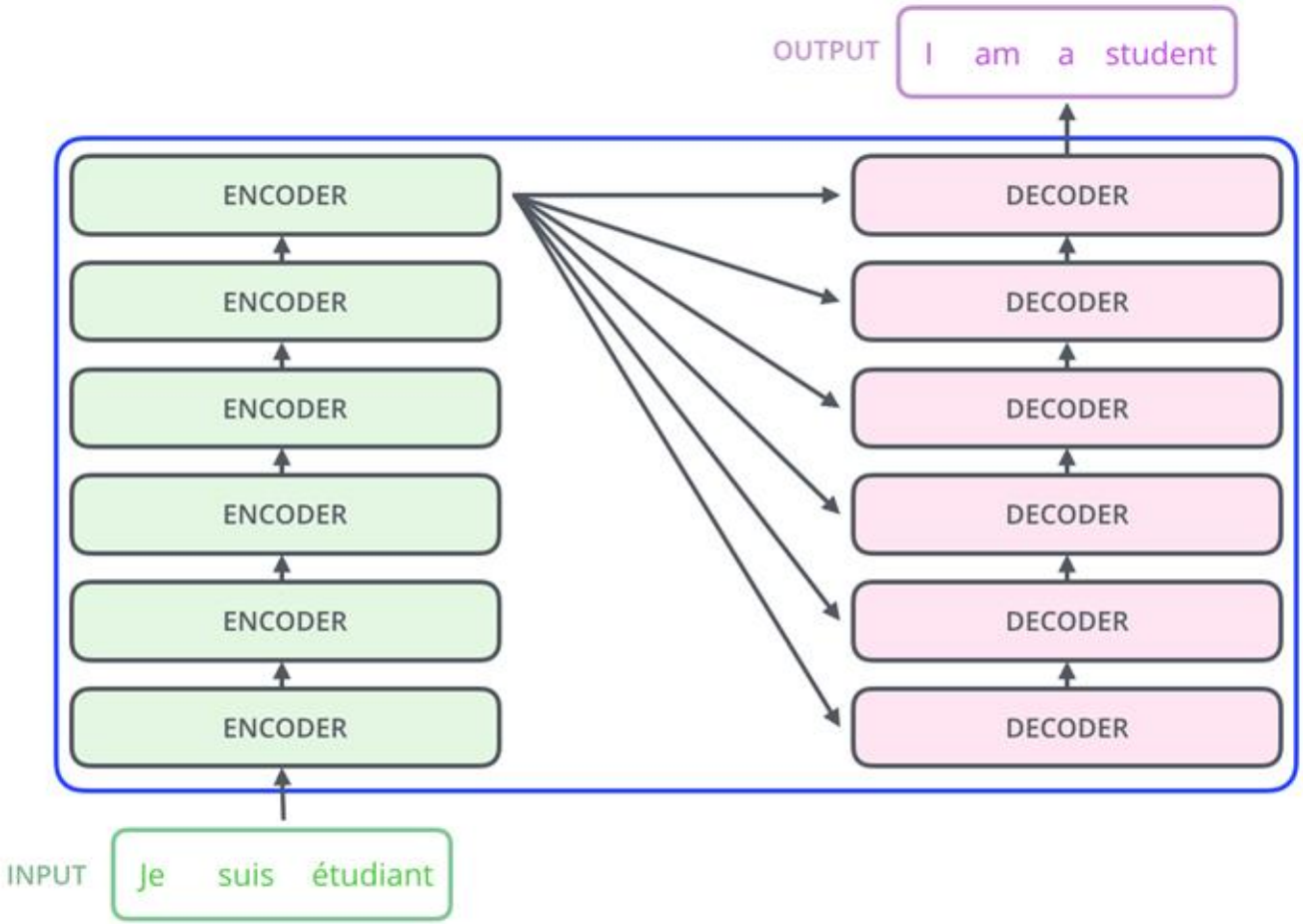
Attention is All You Need

- Transformers
  - A model architecture relying entirely on an attention mechanism to draw global dependencies between input and output
  - Easier to parallelize the computation.

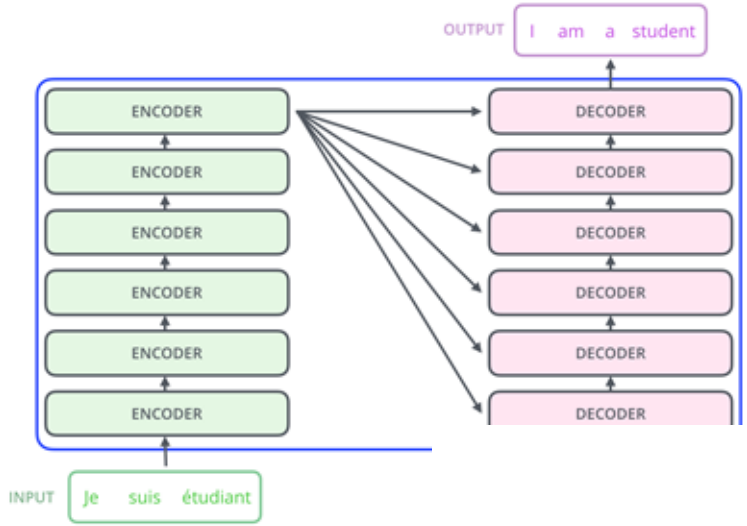


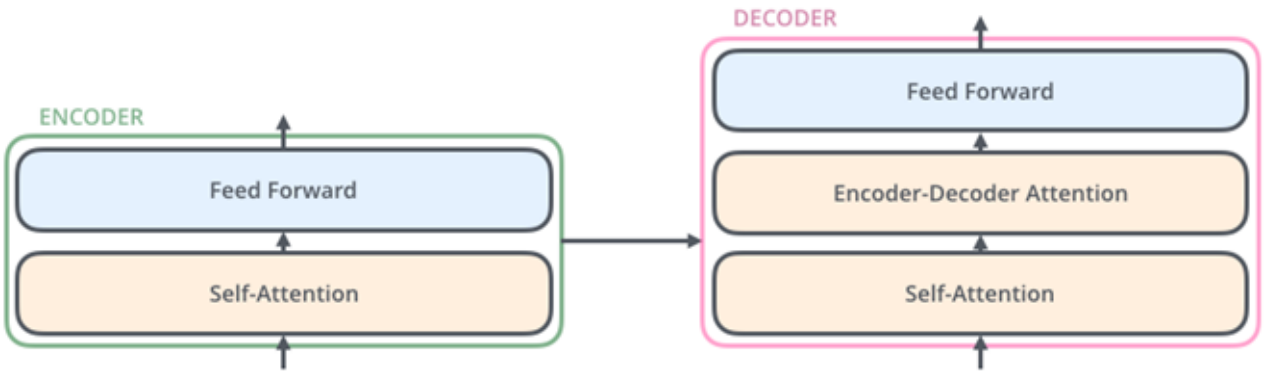
Most of the following visualizations are from http://jalammar.github.io/illustrated-transformer/

Stacks of encoders and decoders



Structures of each encoder and decoder



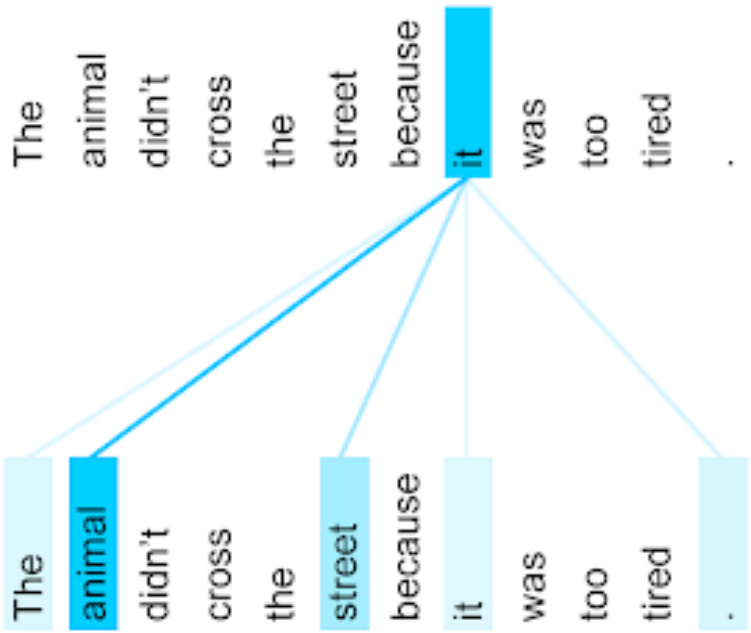


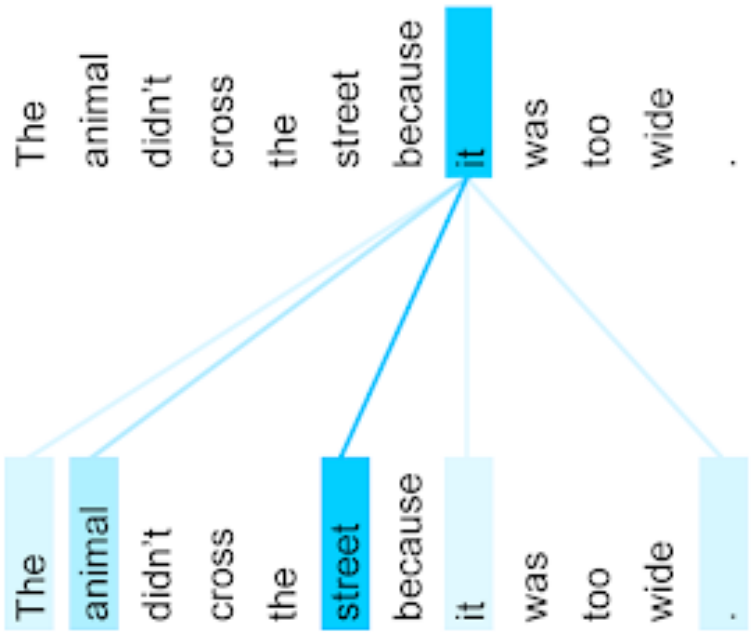
Transformer

Self-attention

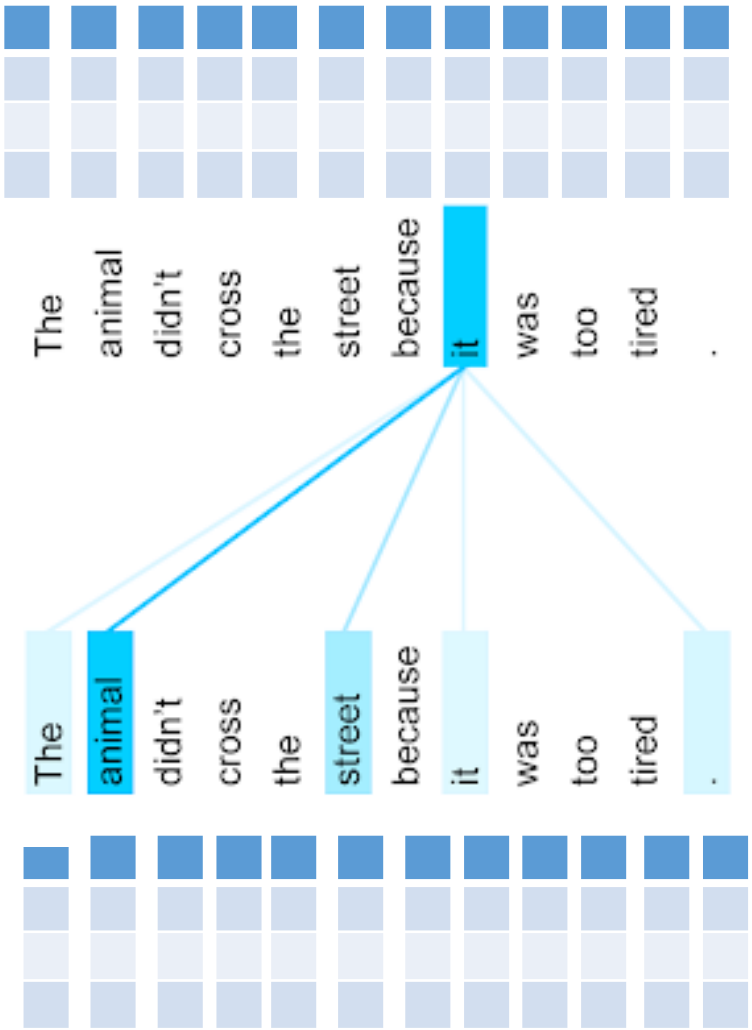
The animal didn't cross the street because it was too tired.

The animal didn't cross the street because it was too wide.





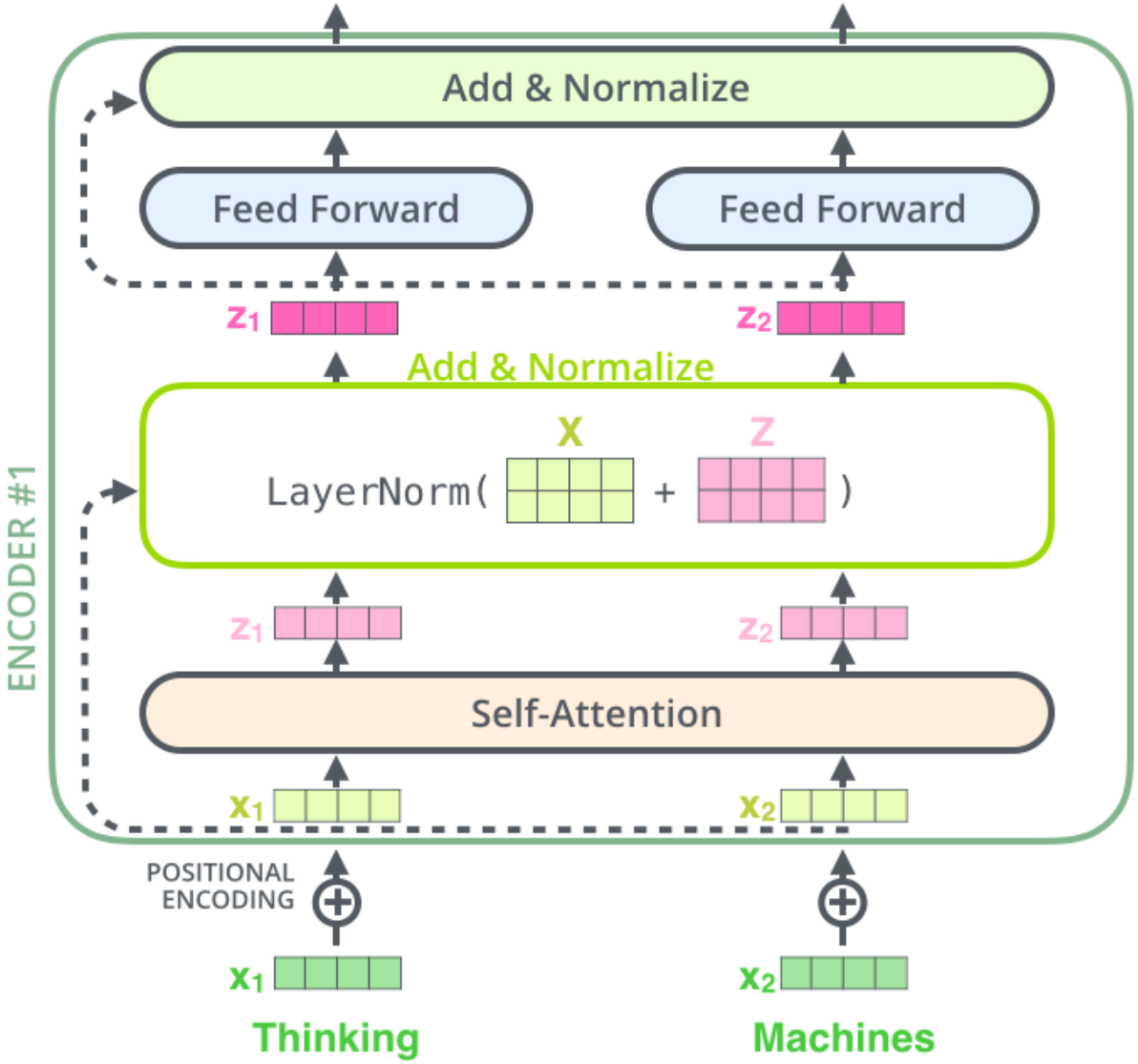
Self-attention (intuition)



Hidden layer H^l

Hidden layer H^{l-1}

Transformer



Self-Attention Layer

$$Q = H^{l-1} W_l^Q$$
$$K = H^{l-1} W_l^K$$
$$V = H^{l-1} W_l^V$$

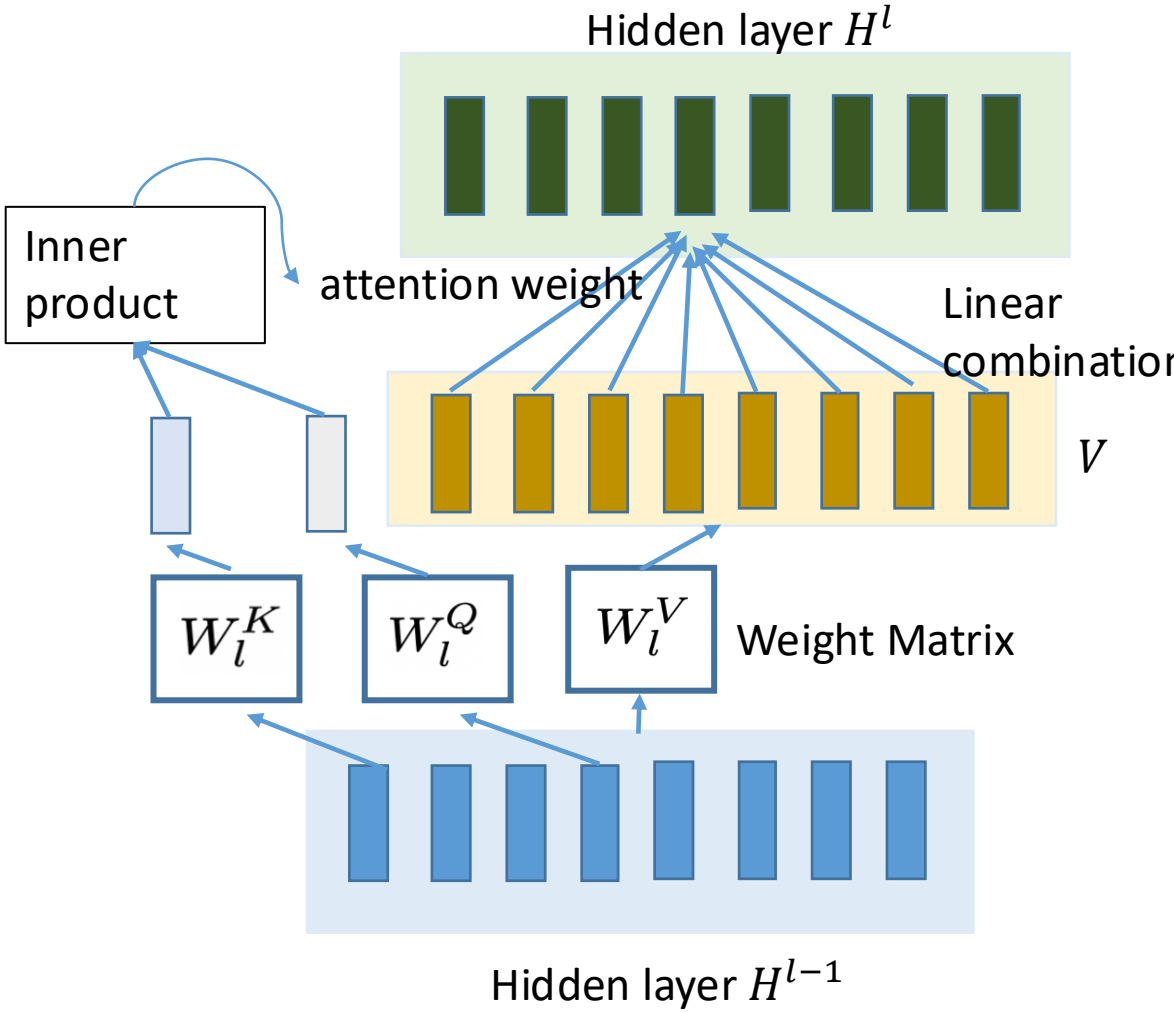
Query

Key

Value

$$\text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) V,$$

a constant

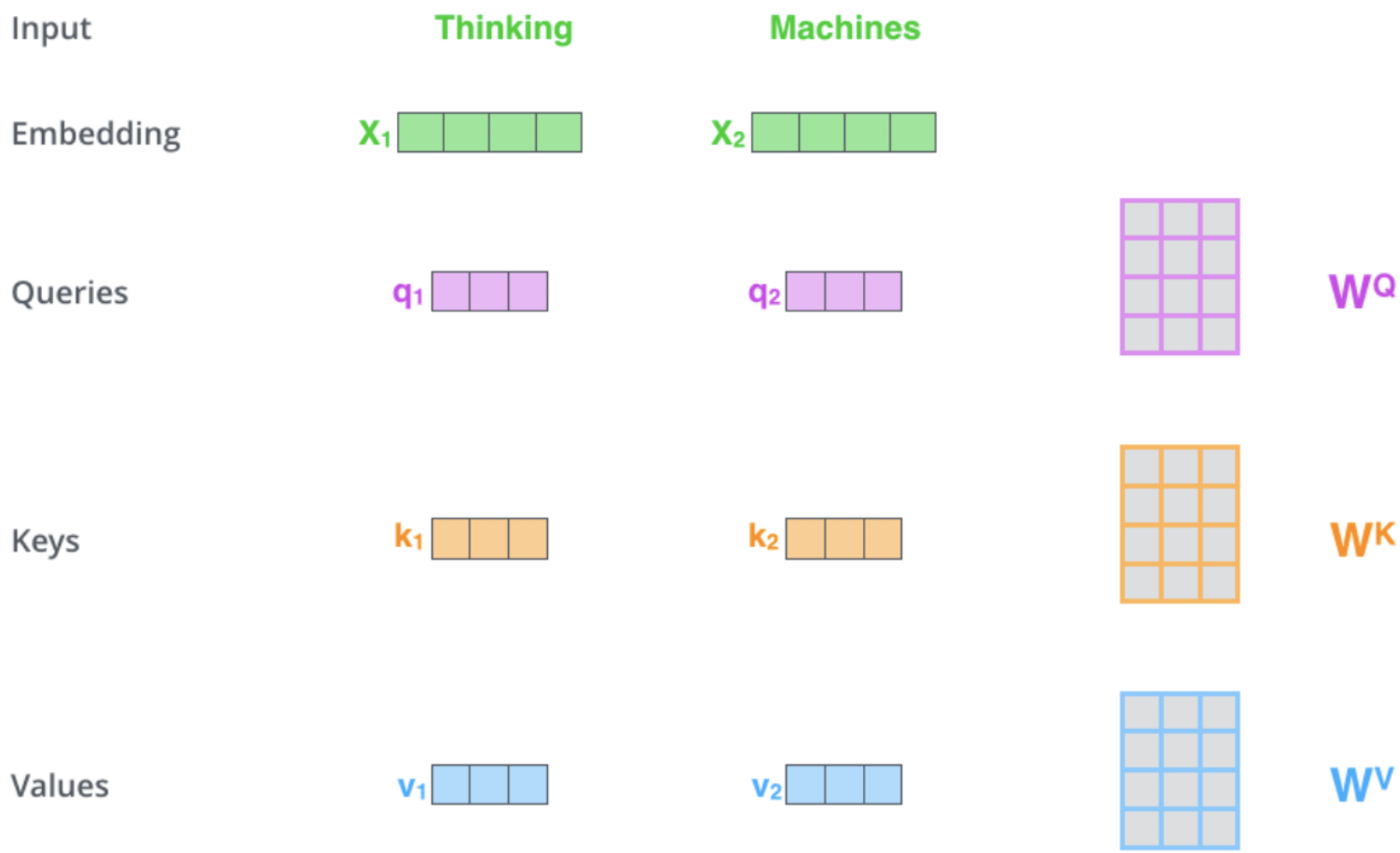


https://poloclub.github.io/transformer-explainer/

Self-Attention

http://nlp.seas.harvard.edu/2018/04/03/attention.html
http://jalammar.github.io/illustrated-transformer/

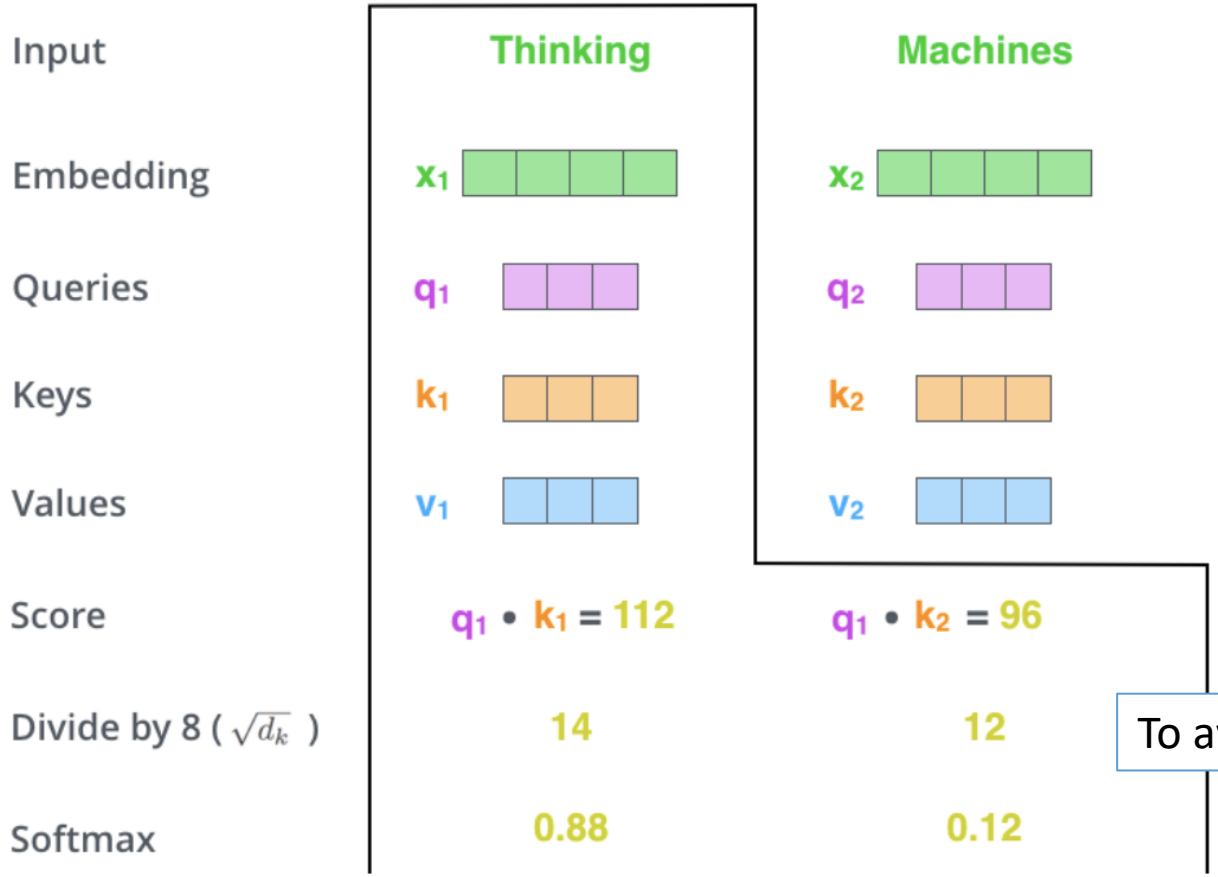
Step 1: break a word vector into three representations



UCLA Computer

Self-Attention

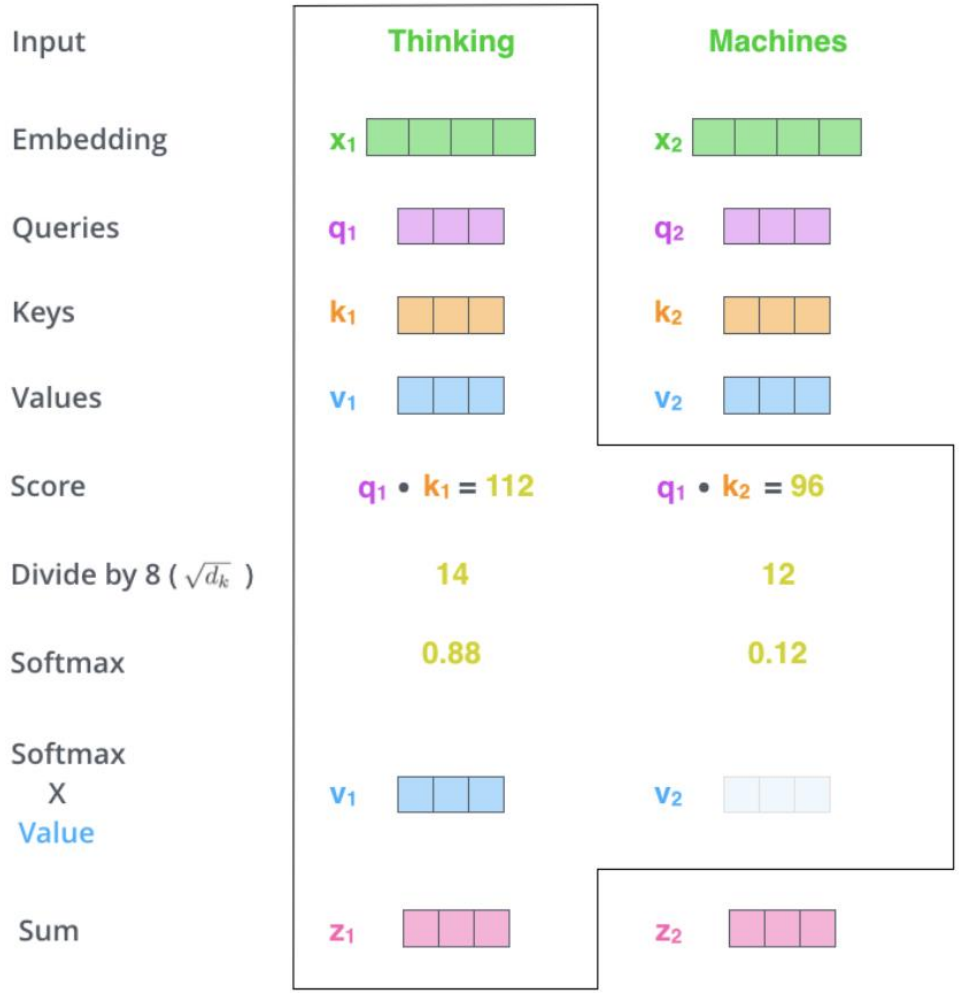
- Step 2: compute pairwise score



To avoid small gradient

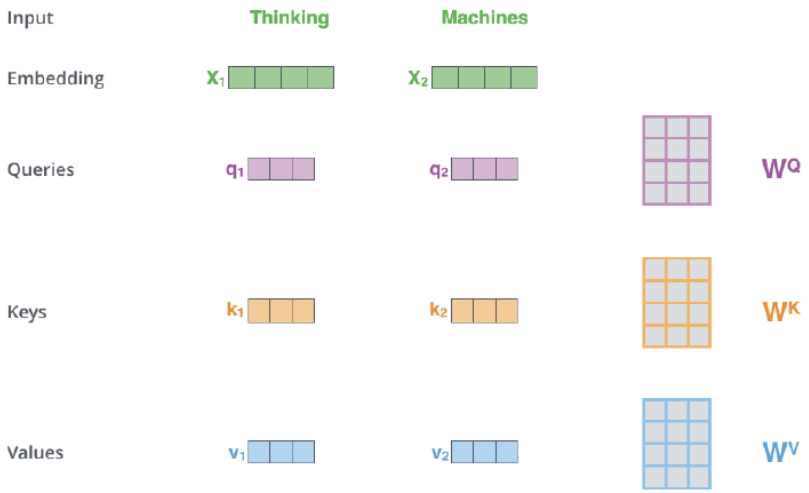
Self-Attention

- Step 3: final vector = linear combination of value vectors



Do it using matrix

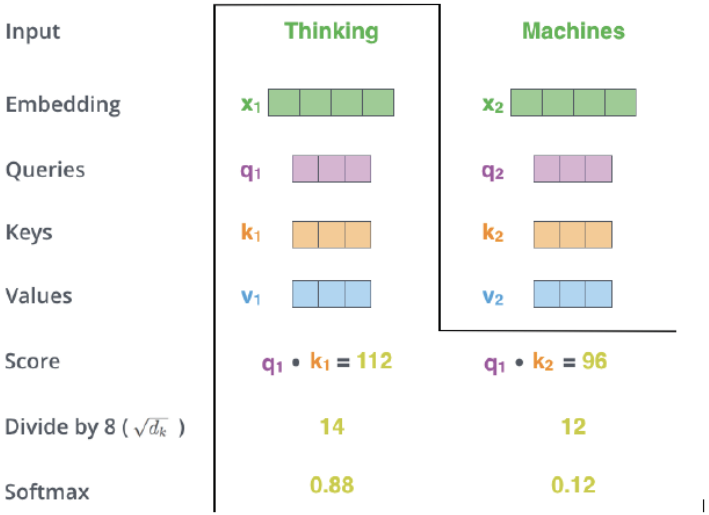
- First step:

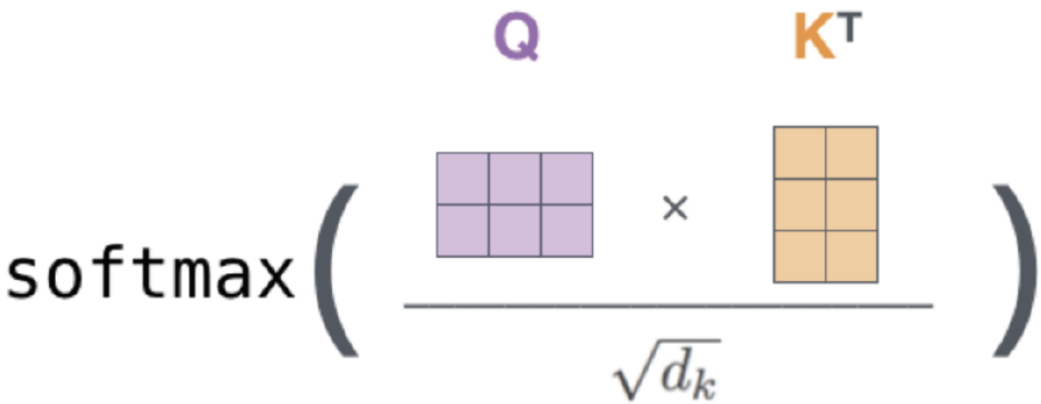


UCLA ENGINEERING
Computer Science

Do it using matrix

- Second step:





A closer look into the attention matrix

In this simple case we consider the sequence length **seq** = 6 and $\mathbf{d_k}$= 512.

$$\text{softmax}\left(\frac{\mathbf{Q}_{(6,512)} \times \mathbf{K}^T_{(512,6)}}{\sqrt{512}}\right) =$$

	YOUR	CAT	IS	A	LOVELY	CAT	Σ
YOUR	0.268	0.119	0.134	0.148	0.179	0.152	1
CAT	0.124	0.278	0.201	0.128	0.154	0.115	1
IS	0.147	0.132	0.262	0.097	0.218	0.145	1
A	0.210	0.128	0.206	0.212	0.119	0.125	1
LOVELY	0.146	0.158	0.152	0.143	0.227	0.174	1
CAT	0.195	0.114	0.203	0.103	0.157	0.229	1

(6, 6)

A closer look into the attention calculation

	YOUR	CAT	IS	A	LOVELY	CAT
YOUR	0.268	0.119	0.134	0.148	0.179	0.152
CAT	0.124	0.278	0.201	0.128	0.154	0.115
IS	0.147	0.132	0.262	0.097	0.218	0.145
A	0.210	0.128	0.206	0.212	0.119	0.125
LOVELY	0.146	0.158	0.152	0.143	0.227	0.174
CAT	0.195	0.114	0.203	0.103	0.157	0.229

(6, 6)

X

V

(6, 512)

=

Attention

(6, 512)

Each row in this matrix captures not only the meaning (given by the embedding) or the position (represented by the positional encodings) of the word, but also contextualized information.

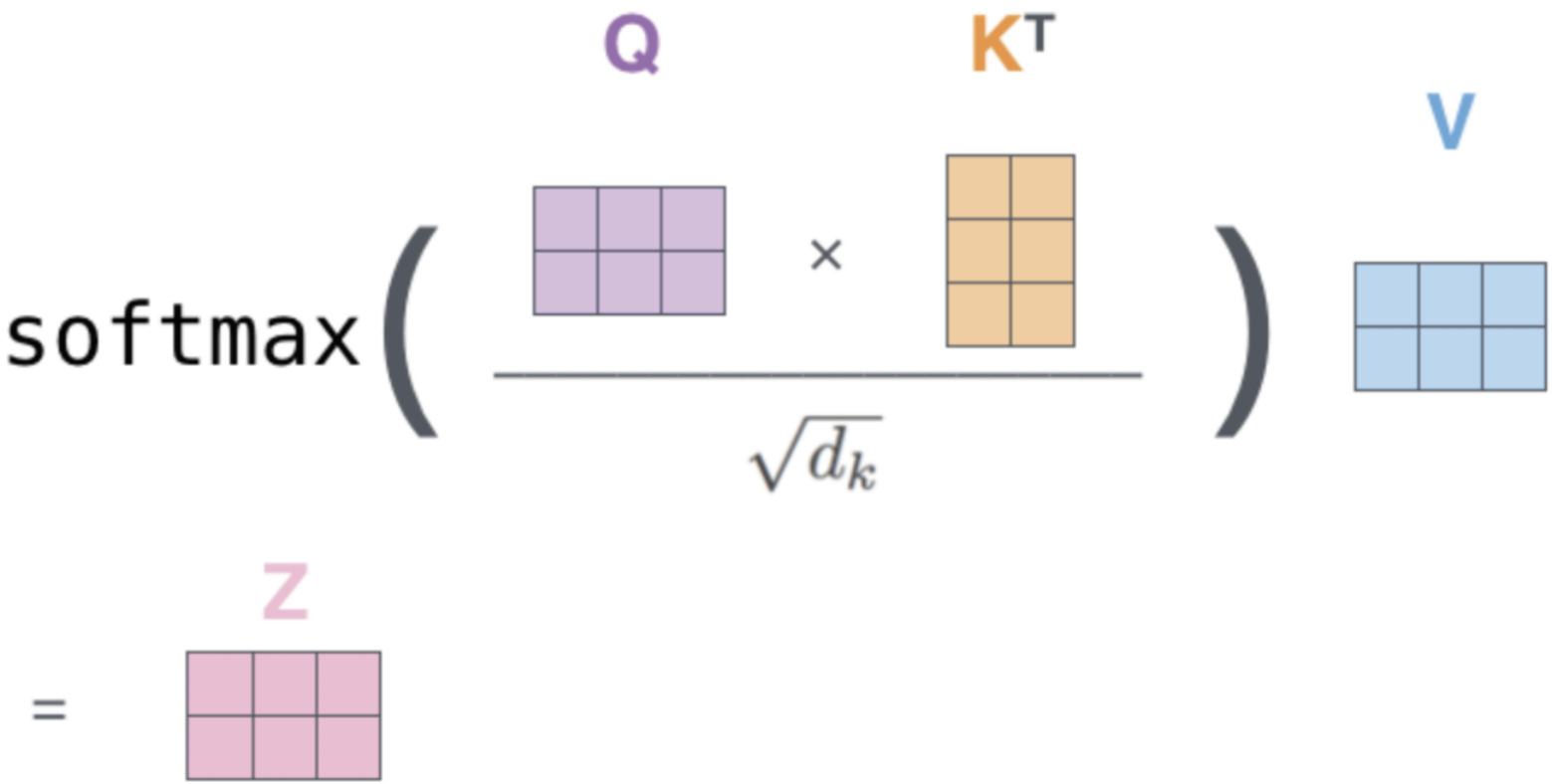
Exercise

- $Q = \begin{Bmatrix} 1 & 1 \\ 0 & 1 \end{Bmatrix}$, $K = \begin{Bmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 1 \end{Bmatrix}$

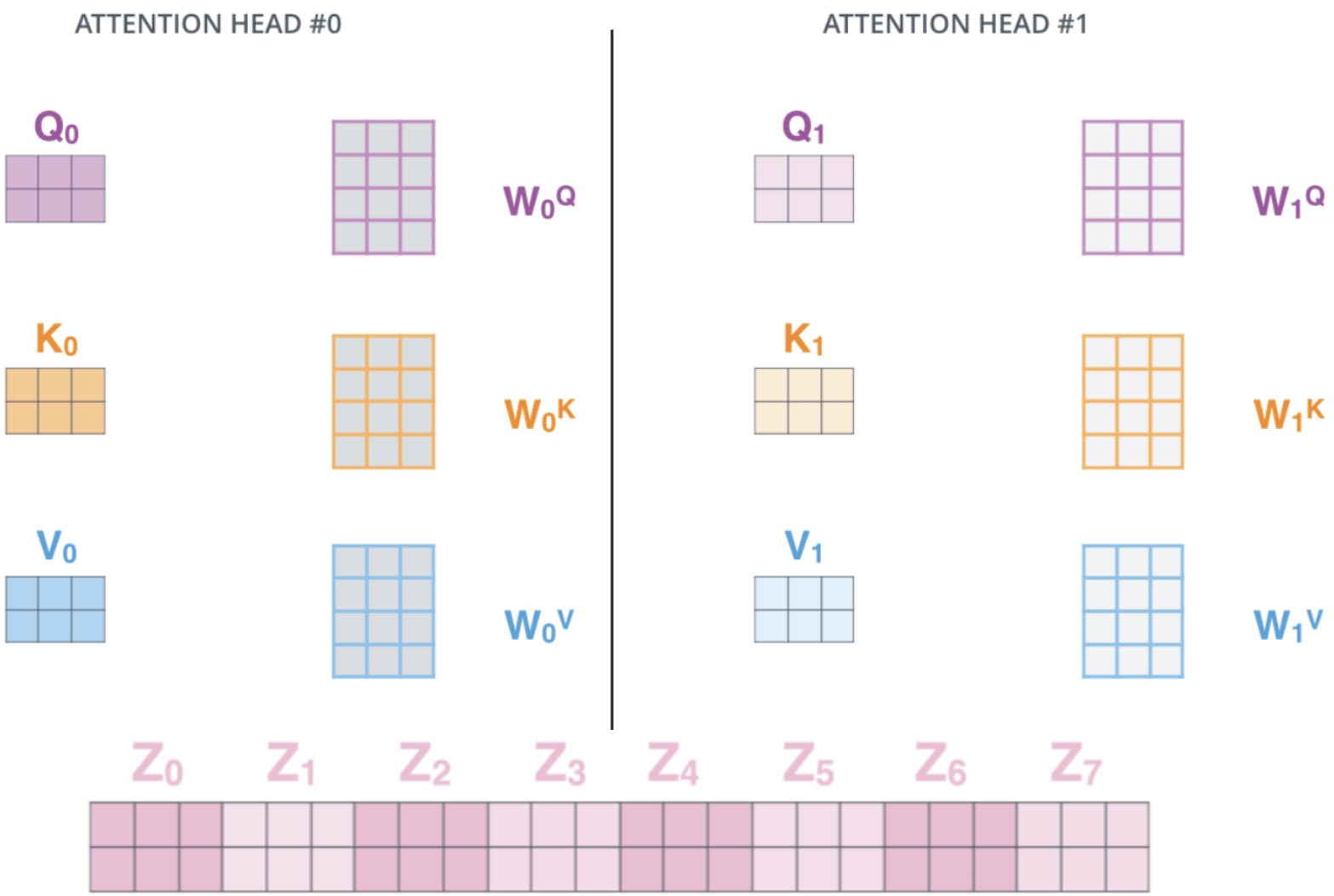
- Attention score = ?

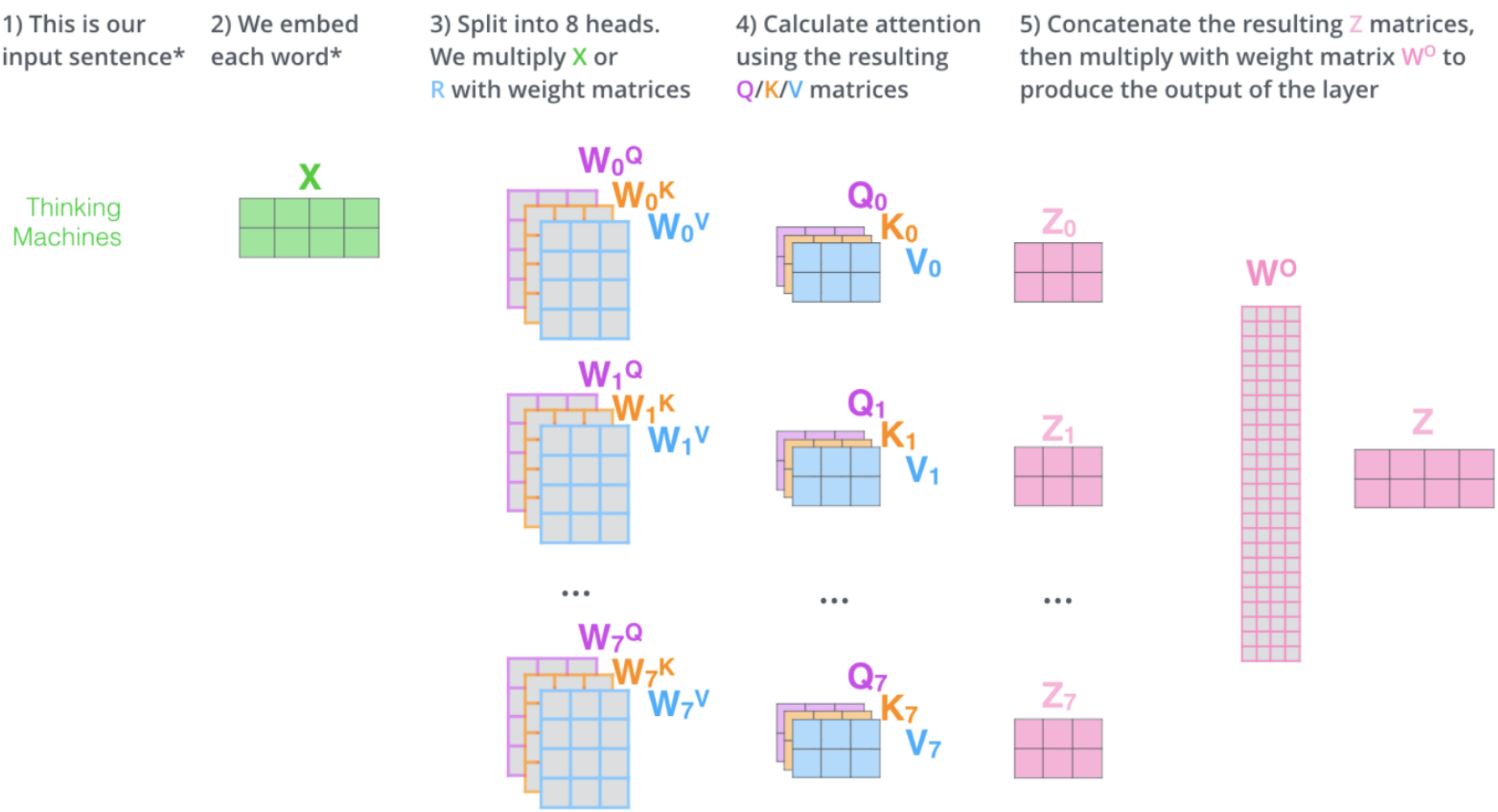
Do it using matrix

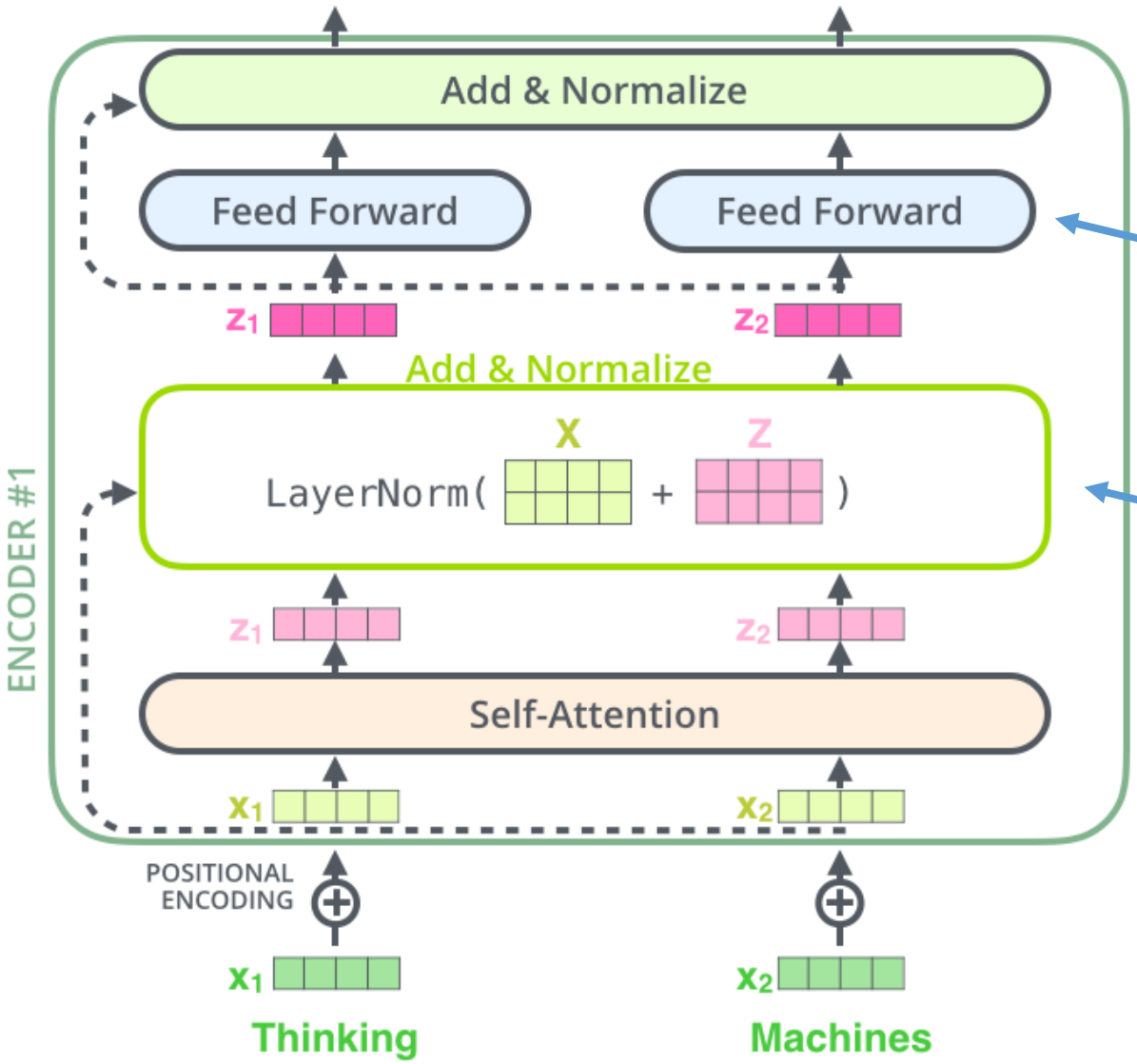
- Third step:



Multiple heads



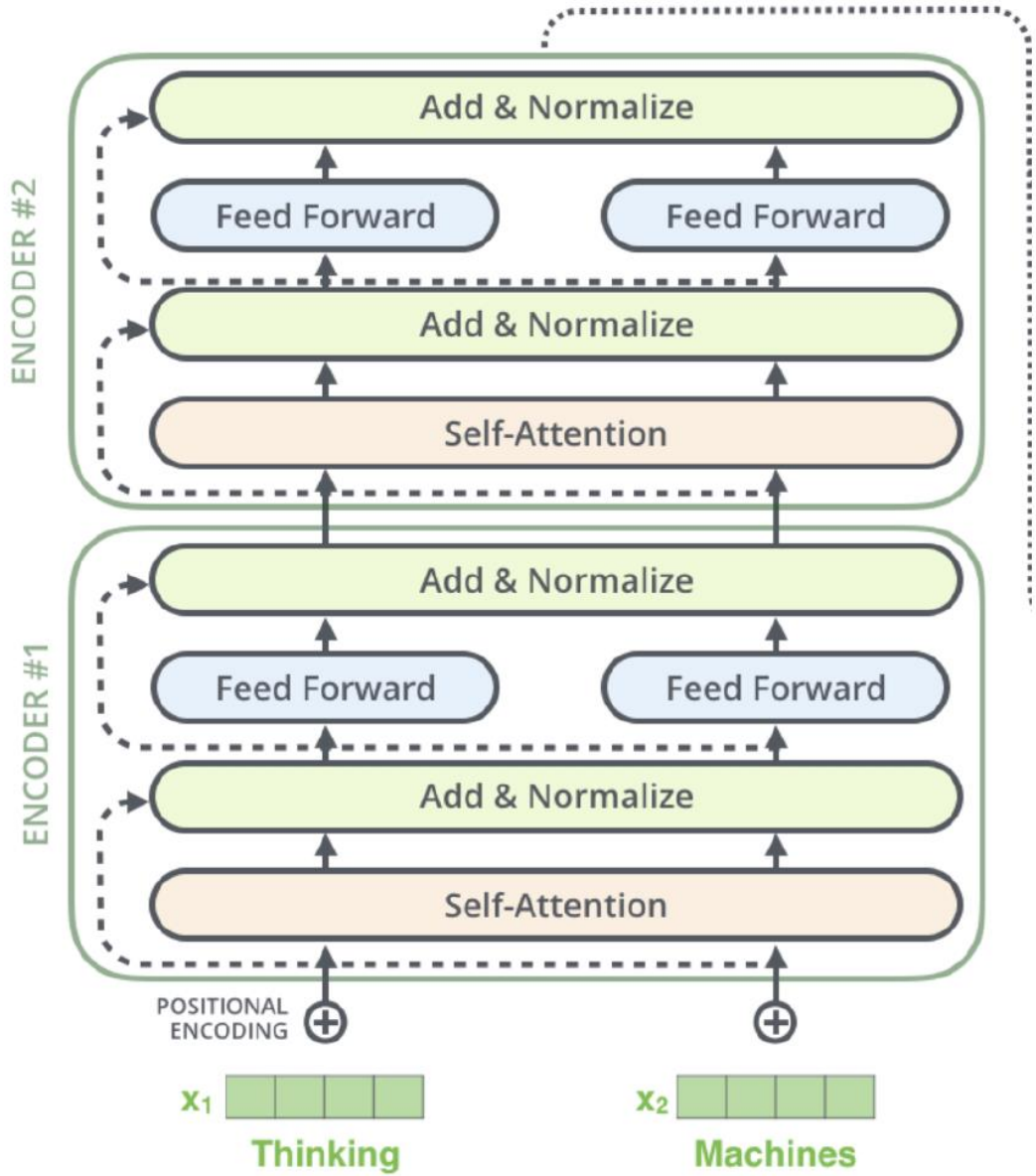

1) This is our input sentence*
2) We embed each word*
3) Split into 8 heads. We multiply X or R with weight matrices
4) Calculate attention using the resulting Q/K/V matrices
5) Concatenate the resulting Z matrices, then multiply with weight matrix W^O to produce the output of the layer
Thinking
Machines
X
W0Q
W0K
W0V
Q0
K0
V0
Z0
W1Q
W1K
W1V
Q1
K1
V1
Z1
...
W7Q
W7K
W7V
Q7
K7
V7
Z7
WO
Z



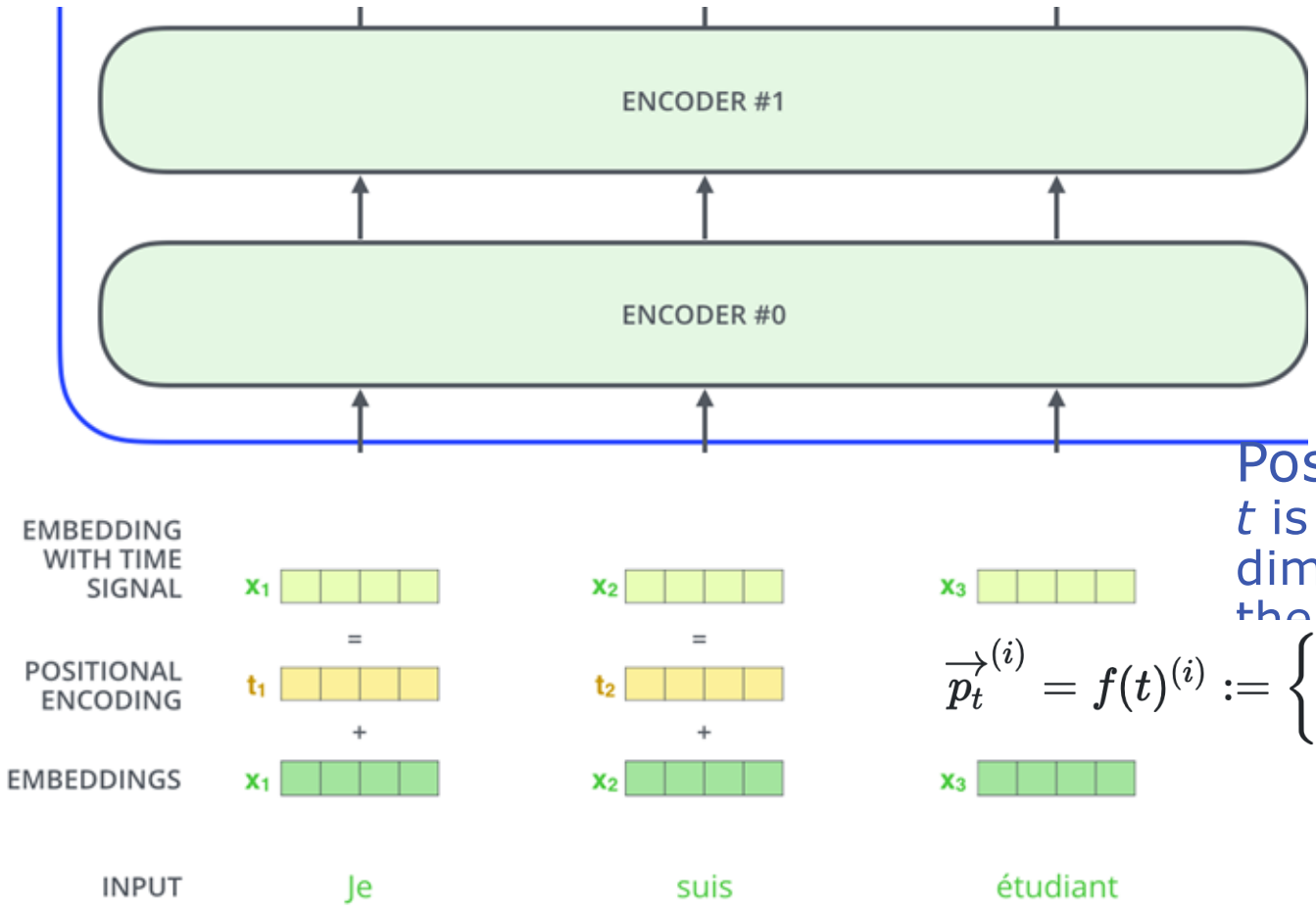
2-layer FF with ReLU

Layernorm changes input features to have mean 0 and variance 1 per layer

Stack multiple layers



Position Encoding



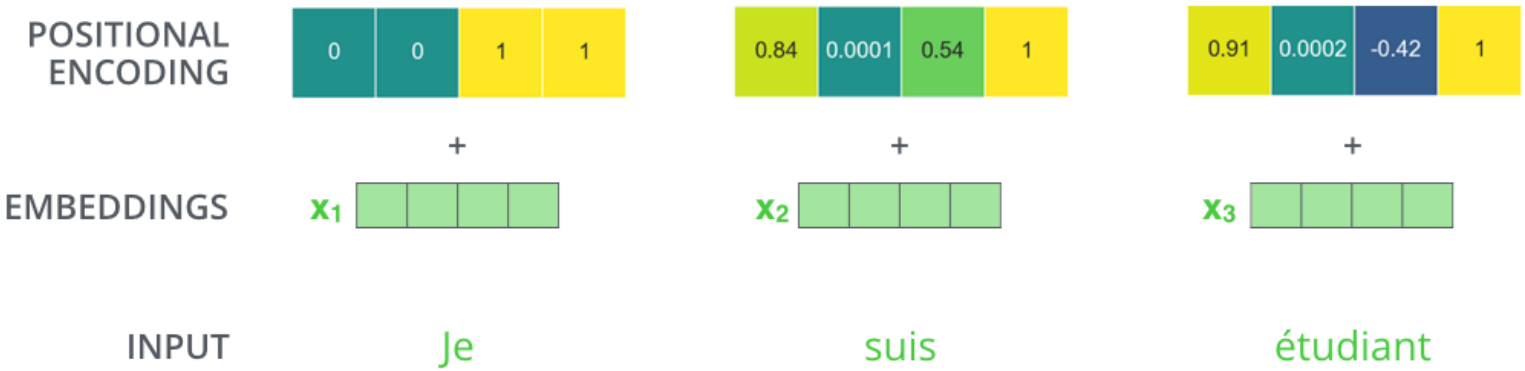
Positional Encoding

t is position, *i* is dimension index, d is the vector size

$$\overrightarrow{p_t}^{(i)} = f(t)^{(i)} := \begin{cases} \sin(\omega_k . t), & \text{if } i = 2k \\ \cos(\omega_k . t), & \text{if } i = 2k+1 \end{cases}$$

$$\omega_k = \frac{1}{10000^{2k/d}}$$

Position Embedding Visualization



Different position embedding strategies

- Original Transformer paper: Sinusoidal embedding
  - Used in all legacy decoder-only large-pretrained models (e.g., GPT-2/3).
  - GPT4 didn't disclose its position embedding details, but they cited rotary position embedding (it is believed that they use RoPE).
- Learned position embedding:
  - Used in encoder only large-pretrained transformer models, such as BERT, RoBERTa, ALBERT, DistilBERT, ERNIE
- Rotary position embedding (RoPE):
  - Used in all newer (open) decoder-only large-pretrained models, such as LlaMA-1/2/3, GPT-J, Pythia, Mistral, and probably in

Residual Connections and Layer Norm

- Issue of information loss -- Self-attention can decide not to attend to itself

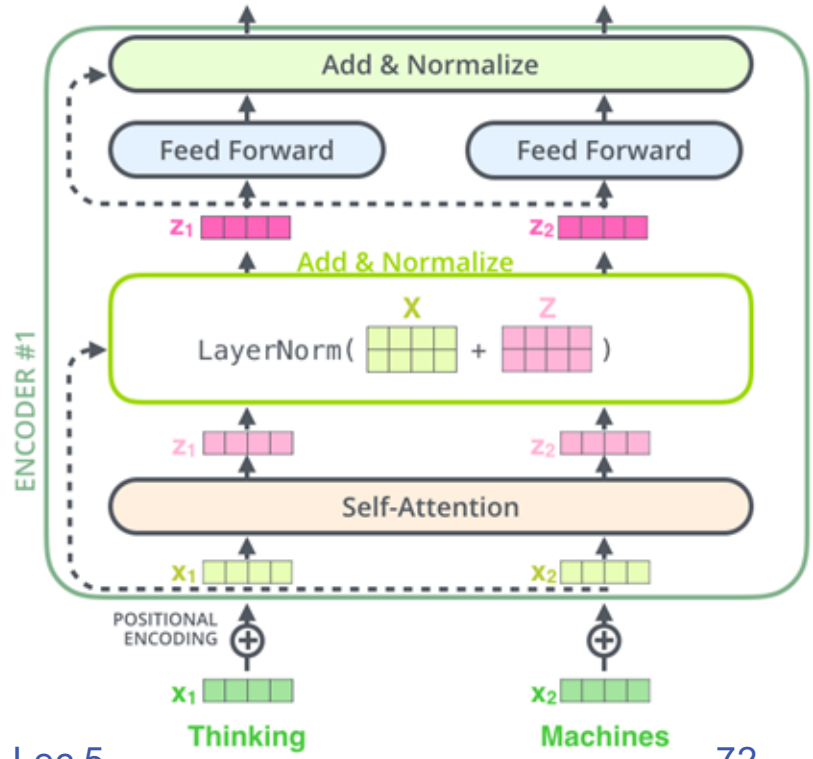
-> Add input back again in each sublayer!

Residual Connection

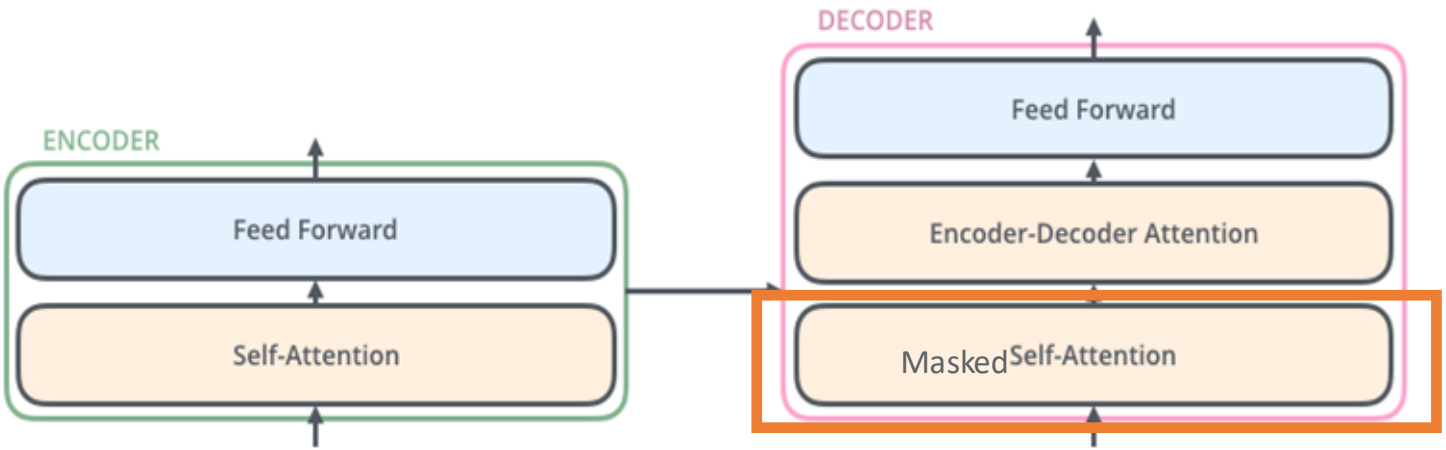
$$\text{LayerNorm}(x + \text{Sublayer}(x))$$

$$\text{LayerNorm}(v) = \gamma \frac{v-\mu}{\sigma} + \beta$$

μ, σ are the mean and standard deviation of the elements in v. Scale γ and bias vector β are parameters.



Transformer Decoders



Masked Self-Attention

- The goal is to make the model *causal*: it means the output at a certain position can only depend on the words in the left-side context. The model **must not** be able to see future words.

	YOUR	CAT	IS	A	LOVELY	CAT
YOUR	0.268	~~0.119~~	~~0.134~~	~~0.148~~	~~0.179~~	~~0.152~~
CAT	0.124	0.278	~~0.201~~	~~0.128~~	~~0.154~~	~~0.115~~
IS	0.147	0.132	0.262	~~0.097~~	~~0.218~~	~~0.145~~
A	0.210	0.128	0.206	0.212	~~0.119~~	~~0.125~~
LOVELY	0.146	0.158	0.152	0.143	0.227	~~0.174~~
CAT	0.195	0.114	0.203	0.103	0.157	0.229

Why?

How?

For what tasks will we need this masked self-attention?

Masked Self-Attention

- Attention masks

0	-inf	-inf	-inf	-inf	-inf
0	0	-inf	-inf	-inf	-inf
0	0	0	-inf	-inf	-inf
0	0	0	0	-inf	-inf
0	0	0	0	0	-inf
0	0	0	0	0	0

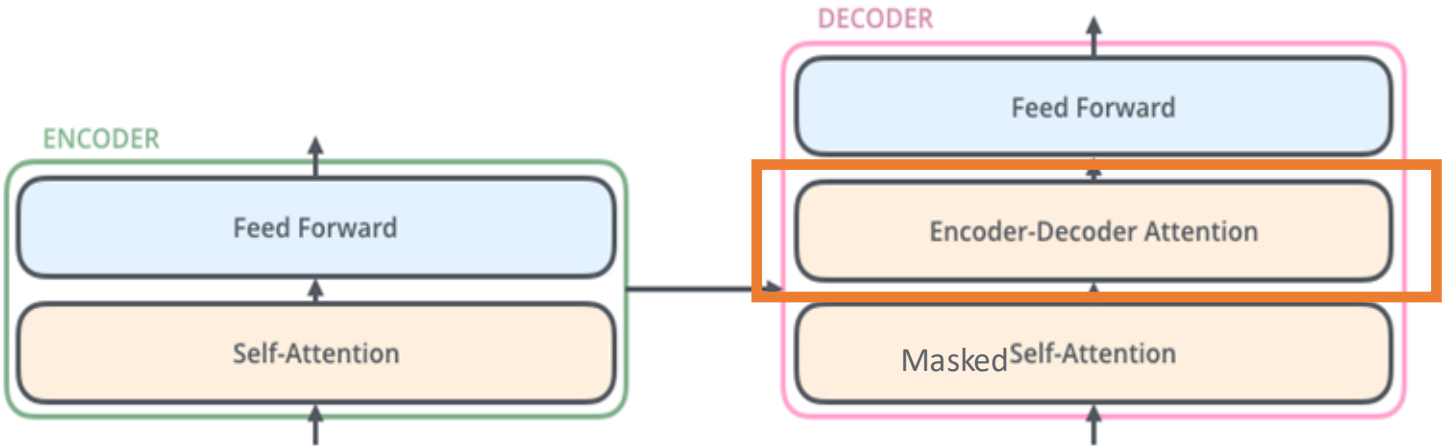
Add to the QK^T attention BEFORE softmax.

What's the effect?

1	0	0	0	0	0
1	1	0	0	0	0
1	1	1	0	0	0
1	1	1	1	0	0
1	1	1	1	1	0
1	1	1	1	1	1

Can we multiply this to the QK^T attention AFTER softmax?

Transformer Decoders



Exercise

How can we design the mask matrix M in Transformer so that, when encoding “love” in “I love to study NLP,” the attention only considers “I” and “study”?

$$Attention(Q, K, V, M, d_k) = softmax(\frac{QK^T + M}{\sqrt{d_k}})V$$

- M will be set to positive infinity for the cells in the attention matrix that are associated with (love, love), (love, to), (love, NLP).
- M will be set to negative infinity for the cells in the attention matrix that are associated with (love, love), (love, to), (love, NLP).
- M will be set to positive infinity for the cells in the attention matrix that are associated with (love, I), (love, study).
- M will be set to zero for the cells in the attention matrix that are associated with (love, love), (love, to), (love, NLP).